



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Aplicación Del Procesamiento Del Lenguaje Natural Cuántico

Autor

Daniel Terés Martínez

Directores

Manuel Pegalajar Cuéllar

Serafín Moral García



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

—
5 de abril de 2026

Aplicación Del Procesamiento Del Lenguaje Natural Cuántico

Daniel Terés Martínez

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Yo, **Daniel Terés Martínez**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 41542052G, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Daniel Terés Martínez

Granada 5 de abril de 2026.

D. **Manuel Pegalajar Cuéllar**, Profesor del Área de XXXX del Departamento Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

D. **Serafín Moral García**, Profesor del Área de XXXX del Departamento Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Aplicación Del Procesamiento Del Lenguaje Natural Cuántico***, ha sido realizado bajo su supervisión por **Daniel Terés Martínez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada 5 de abril de 2026.

Los directores:

Manuel Pegalajar Cuéllar Serafín Moral García

Agradecimientos

Poner aquí agradecimientos...

Índice general

1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	3
1.3. Estructura del documento	3
2. Números complejos	5
2.1. ¿Qué es un número complejo?	5
2.2. Representación binómica	5
2.3. Representación geométrica en $\mathbb{R}^2$	6
2.4. Representación forma polar	7
2.4.1. Operaciones	8
2.5. Representación en forma exponencial (Euler)	9
3. Notación Dirac	11
3.1. Notación bra-ket	11
4. ¿Qué es un Qubit?	13
4.1. ¿Cómo expresarlo con notación Dirac?	13
4.2. Esfera de Bloch	14
4.3. Puertas cuánticas	16
5. Procesamiento Lenguaje Natural (PLN)	19
5.1. ¿Qué es el procesamiento de lenguaje natural?	19
5.2. Fases del PLN	19
5.2.1. Preprocesamiento y Tokenización	20
5.2.2. Análisis sintáctico	21
5.2.3. Análisis semántico	21
5.2.4. Integración del discurso	22
5.2.5. Análisis pragmático	22
6. Procesamiento del Lenguaje Natural Cuántico (PLNC)	23
6.1. Motivación para el uso de PLNC	23
6.2. Codificación de información clásica en estados cuánticos	24

6.2.1. Técnicas de codificación de datos clásicos a estados cuánticos	25
6.3. Circuitos Variacionales Cuánticos (CVC) como modelos de aprendizaje	27
7. Representación Vectorial de Palabras (Word Embeddings)	29
7.1. Introducción al concepto de <i>Embedding</i>	29
7.1.1. Embeddings de palabras (<i>Word Embeddings</i>)	30
7.2. Mecanismos geométricos de los embeddings	32
7.2.1. Métricas de Distancia	32
7.2.2. Información Latente	34
7.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert . .	36
7.3.1. Codificación multidimensional: Forma y Contenido . .	36
7.3.2. Afinidad Cuántica, Superposiciones y Fases	37
8. Implementación Clásica: Algoritmo Word2Vec	39
8.1. Arquitectura del modelo	39
8.1.1. Modelos de entrenamiento: CBOW y Skip-gram	41
8.2. Función de optimización	45
9. Implementación Cuántica: Algoritmo QWord2Vec	49
9.1. Adaptación del algoritmo al entorno cuántico	49
9.2. Diseño del Circuito Cuántico Parametrizado	49
9.2.1. Estimación de la longitud del circuito	52
9.3. Definición de la función de evaluación y pérdida usadas . . .	54
9.3.1. Evaluación de los datos de embeddings	54
9.3.2. Función de pérdida	55
9.4. Estrategia de optimización	55
9.4.1. Funcionamiento del algoritmo	56
10. Experimentación y Análisis de Resultados	59
10.1. Metodología experimental	59
10.1.1. Dataset y preprocesamiento	60
10.1.2. Entorno de ejecución y herramientas	61
10.2. Configuración de hiperparámetros	62
10.2.1. Hiperparámetros de Word2Vec	62
10.2.2. Hiperparámetros de QWord2Vec	64
10.3. Entrenamiento	66
10.3.1. Word2Vec	66
10.3.2. QWord2Vec	69
10.4. Evaluación de los embeddings	72
10.4.1. Similitud semántica entre palabras	72
10.5. Análisis de tiempos de ejecución	72

Bibliografía

73

Glosario

77

Capítulo 1

Introducción

La computación cuántica ha experimentado un avance notable en las últimas décadas, posicionándose como una de las tecnologías con mayor potencial frente a los sistemas de computación clásicos. En particular, el campo del Quantum Machine Learning (QML) está creciendo de forma acelerada, con aplicaciones en problemas de clasificación, modelos generativos y regresión, entre otros dominios que también abarca el Machine Learning (ML) clásico [4, 3, 24]. Los expertos en estos campos apuntan a que muchos de los resultados obtenidos hasta la fecha mejoran la velocidad de entrenamiento así como la capacidad de generalización.

De forma paralela, el campo de la Inteligencia Artificial (IA) ha sido sacudido por la irrupción de los Large Language Models (LLMs), como ChatGPT, Gemini o Claude entre otros, que han demostrado una capacidad sin precedentes para procesar y generar lenguaje humano. En el centro de estos modelos se encuentran los *word embeddings* que son representaciones vectoriales de palabras que codifican sus propiedades semánticas y sintácticas en un espacio continuo. Modelos como Word2Vec [23], publicado por Google en 2013, sentaron las bases de esta técnica y prácticas de estos modelos, tanto que es uno de los componentes principales que hacen que su motor de búsqueda sea tan efectivo. El modelo que propuso, sigue siendo una referencia fundamental en el Procesamiento del Lenguaje Natural (PLN).

La intersección entre ambas disciplinas ha dado lugar al Procesamiento del Lenguaje Natural Cuántico (PLNC), una línea de investigación que busca aprovechar los fenómenos de la cuántica, como la superposición y el entrelazamiento, para representar y procesar el lenguaje de manera más eficiente. Aunque se han propuesto modelos de inspiración cuántica para recuperación de información [44, 45] y para embeddings de palabras [19], hasta hace muy poco (menos de 1 año), no existía ningún circuito cuántico diseñado específicamente para aprender representaciones de palabras, es decir, un equivalente cuántico de Word2Vec.

Este trabajo de fin de grado realizado se adentra en investigar si con

algoritmos ya existentes, se puede mejorar en cuanto a representatividad la primera versión del modelo cuántico de Word2Vec (QWord2Vec). El paper de referencia principal utilizado para el desarrollo de este modelo, es el de Ohno [24], en el que se propone *QWord2Vec*, un circuito cuántico parametrizado que adapta la arquitectura de Word2Vec al entorno cuántico. Dicho circuito, entrenado mediante el optimizador Quantum Natural Simultaneous Perturbation Stochastic Approximation (QN-SPSA) y una función de pérdida que combina la entropía cruzada con el coeficiente de correlación de Pearson, aprende embeddings cuánticos de palabras a partir de un corpus de texto muy pequeño (debido a las limitaciones físicas de utilizar simuladores cuánticos).

1.1. Motivación

La motivación principal de este trabajo es explorar si un modelo cuántico es capaz de aprender relaciones semánticas entre palabras con una calidad comparable a la del modelo clásico Word2Vec. Dicho modelo, que constituye el punto de referencia de este trabajo, lleva más de una década consolidado como uno de los estándares fundamentales del PLN. La pregunta central que se pretende responder es, por tanto: ¿puede el espacio de Hilbert, con su capacidad para representar estados en superposición y aprovechar el entrelazamiento cuántico, servir como espacio latente para los *embeddings* de palabras con resultados equiparables o superiores a los del espacio euclídeo clásico?

Más allá de esta pregunta, el trabajo ofrece también la oportunidad de estudiar en profundidad la frontera actual entre el PLN y la computación cuántica, dos de los campos más dinámicos de la ciencia computacional contemporánea. Para ello, se recorren tanto los fundamentos matemáticos imprescindibles (notación de Dirac, qubits y circuitos cuánticos parametrizados) como los fundamentos del PLN clásico (representaciones vectoriales de palabras y Word2Vec), hasta alcanzar su convergencia en el modelo QWord2Vec.

A diferencia del trabajo de referencia [24], cuyo objetivo era entrenar ambos modelos bajo condiciones lo más similares posible para evaluar en qué medida la versión cuántica reproducía las representaciones de la clásica, en este trabajo se adopta una perspectiva distinta. Se entrena cada modelo con las mejores técnicas disponibles en su campo respectivo, con el fin de determinar cuál ofrece resultados más competitivos en las condiciones actuales del estado del arte.

Esta aproximación no cuestiona la dirección de la investigación en PLNC, sino que la complementa planteando una pregunta diferente ¿cuán madura se encuentra hoy la computación cuántica para competir con una técnica clásica tan consolidada como Word2Vec en el campo de PLN? Dado que QWord2Vec

fue propuesto hace menos de un año, los resultados de este trabajo deben interpretarse como un diagnóstico del estado actual del campo, y no como una valoración definitiva de su potencial futuro.

1.2. Objetivos

Los objetivos concretos de este trabajo son:

1. Estudiar y comprender los fundamentos de la computación cuántica necesarios para diseñar e implementar circuitos cuánticos parametrizados.
2. Estudiar el algoritmo Word2Vec clásico y su variante Skip-gram, analizando su arquitectura y proceso de entrenamiento.
3. Comprender la propuesta de QWord2Vec [24]: su circuito cuántico parametrizado, la función de pérdida empleada y la estrategia de optimización que usó.
4. Implementar ambos modelos (Word2Vec y QWord2Vec) en Python, utilizando las librerías Gensim y Qiskit respectivamente, entrenando cada uno con las mejores técnicas disponibles en su paradigma.
5. Evaluar la calidad de los *embeddings* generados por QWord2Vec y compararlos con los producidos por Word2Vec mediante el coeficiente de correlación de Pearson.
6. Proponer y evaluar modificaciones sobre la función de optimización y la inicialización de parámetros del modelo QWord2Vec, analizando su impacto en el entrenamiento y en la calidad de los *embeddings* obtenidos.

1.3. Estructura del documento

El resto del documento se organiza de la siguiente manera. En primer lugar, se presentan los fundamentos matemáticos imprescindibles: el **Capítulo 2** introduce los números complejos, y el **Capítulo 3** describe la notación de Dirac, ambos necesarios para trabajar con estados cuánticos. El **Capítulo 4** describe qué es un qubit, cómo se representa y qué operaciones pueden realizarse sobre él.

A continuación, los capítulos de PLN: el **Capítulo 5** introduce el procesamiento del lenguaje natural clásico y sus fases principales; el **Capítulo 6** presenta el procesamiento del lenguaje natural cuántico y su motivación; el **Capítulo 7** describe los word embeddings y el espacio latente; y el **Capítulo 8** explica en detalle el algoritmo Word2Vec y su variante Skip-gram.

El **Capítulo 9** es el núcleo técnico del trabajo: describe la arquitectura del circuito cuántico de QWord2Vec, la función de pérdida empleada y el optimizador QN-SPSA. Finalmente, el **Capítulo 10** recoge el proceso experimental, los resultados obtenidos y el análisis comparativo entre los dos modelos.

Capítulo 2

Números complejos

2.1. ¿Qué es un número complejo?

Un número complejo (representado por el símbolo $\mathbb{C}$) es la combinación de números reales e imaginarios. La parte real se puede expresar por un número entero o sus decimales, la parte imaginaria es aquella cuyo cuadrado es negativo.

Los números complejos sirven para dar sentido a la raíz cuadrada de números negativos. Esto permite que ecuaciones que tengan como resultado una raíz cuadrada negativa, puedan ser resueltas. Con esto se abre la puerta a un gran mundo en el que todas las operaciones (excepto dividir entre 0), son posibles.

Las características principales de los números complejos con las siguientes:

- Tiene distintas formas de expresarse.
- Dos números complejos se consideran iguales cuando tienen mismo componente real e imaginario.
- $\mathbb{C}$ conforma un espacio vectorial de dos dimensiones.
- Los números complejos no pueden mantener un orden. Es decir no se puede encontrar una propiedad $>$ o $<$ que un número para todos los números complejos.

2.2. Representación binómica

Consideremos la unidad imaginaria $i = \sqrt{-1}$ la cual representamos en el punto $(0, 1)$ del plano. Con esto, se puede situar los números $2i, 3i, \dots, -i, -2i, \dots$ en el eje vertical, a estos números que tienen forma bi , los llamaremos imaginarios. Entonces cualquier punto en el plano (a, b) donde $a, b, \in \mathbb{R}$, se puede

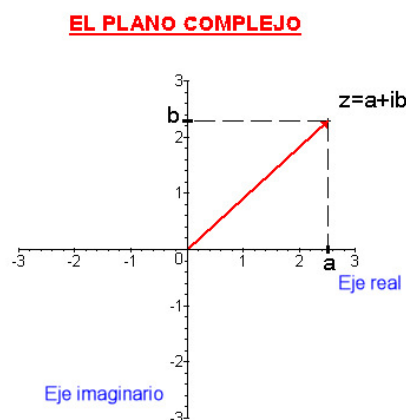


Figura 2.1: Plano sobre como representar número complejo

expresar como $(a, 0) + (0, b)$, siendo la suma de un número real y un número imaginario. Su representación es la siguiente:

$$z = a + bi, \quad a, b \in \mathbb{R}, \quad i = \sqrt{-1}$$

Con esta representación, a es la parte real y b la parte imaginaria. Se puede representar en el plano mediante el punto $Z(a, b)$, a esto se le conoce como **afijo**.

- **Conjugado:** Se define de la forma $\bar{z} = a - ib$.
- **Inverso:** Se define como $z^{-1} = \frac{a}{a^2+b^2} - \frac{b}{a^2+b^2}i$.

Los números complejos se pueden sumar, restar, multiplicar y dividir siguiendo las reglas de las operaciones con números reales.

2.3. Representación geométrica en $\mathbb{R}^2$

Para representar gráficamente un número complejo, debemos dibujarlo en el plano complejo. Está formado por un **eje real** y **eje imaginario**, en el eje real se representa la parte real mientras que en el eje imaginario la parte imaginaria.

Basándonos en la definición del apartado anterior sobre el inverso y conjugado de un número complejo, al representarlo de forma gráfica queda de la siguiente manera:

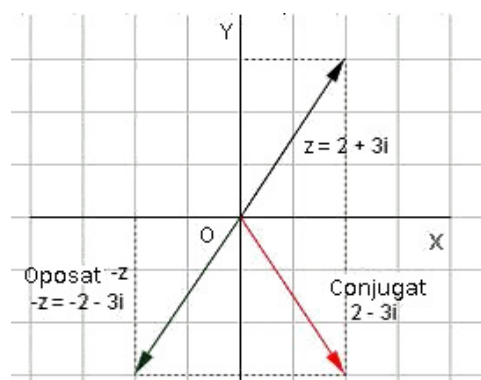


Figura 2.2: Representación en plano sobre conjugado e inverso

2.4. Representación forma polar

Esta representación sirve para destacar los atributos gráficos que tienen los números complejos.

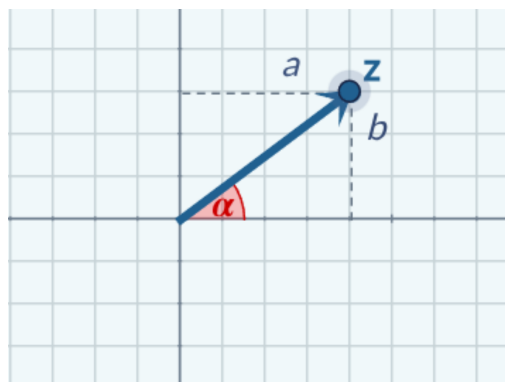


Figura 2.3: Gráfica de la forma polar de un número complejo

- **Módulo (r):** Distancia del afijo al origen de coordenadas.
- **Argumento ($\arg(z)$):** Ángulo que forma el segmento $\overline{OZ}$ con el semieje positivo medido en el sentido contrario de las agujas del reloj entre 0° y 360° .

Con el módulo y el argumento se puede expresar el número complejo z en **forma polar** de la siguiente forma:

$$r_\alpha \quad \text{con } r = |z| \text{ y } \alpha = \arg(z)$$

En base a la forma polar, se puede expresar **z** en **forma trigonométrica** de la siguiente manera:

$$z = r(\cos \alpha + i \operatorname{sen} \alpha),$$

Esto nos puede servir para pasar de forma polar a forma binómica.

2.4.1. Operaciones

Con esta representación, se permite realizar la multiplicación y división de una manera sencilla. Para ello hay que **expresar los números complejos en forma trigonométrica**.

- **Producto:** $r \cdot r' (\cos(\alpha + \beta) + i \operatorname{sen}(\alpha + \beta))$
Se observa que el módulo del número complejo es el resultante del producto de los módulos y que el argumento es la suma de los argumentos.
- **Cociente:** $\frac{r}{r'} (\cos(\alpha - \beta) + i \operatorname{sen}(\alpha - \beta))$ El resultado del cociente es el cociente de los módulos y la diferencia de los argumentos

Por lo que en **forma polar** quedaría de la siguiente manera:

- **Producto:** $r_\alpha \cdot r'_\beta = (r \cdot r')_{\alpha+\beta}$
- **Cociente:** $\frac{r_\alpha}{r'_\beta} = \left(\frac{r}{r'}\right)_{\alpha-\beta}$

Para calcular **potencia** con exponente natural **n**, en forma polar, hay que elevar el módulo a **n** y multiplicar el argumento por **n**.

$$(r_\alpha)^n = (r^n)_{n \cdot \alpha}$$

La **raíz n-esima** de un número complejo en forma polar, es otro número complejo m_β que debe de cumplir:

$$(m_\beta)^n = r_\alpha$$

$$m = \sqrt[n]{r} \quad \text{y} \quad \beta = \frac{\alpha}{n} + \frac{360^\circ * k}{n}$$

Todo número complejo tiene n raíces n-ésimas. Todos los afijos están situados sobre una circunferencia y son los vértices de un polígono regular de n lados.

Un ejemplo de forma gráfica calculando la raíz n-esima con lo explicado anterioremente, es el siguiente:

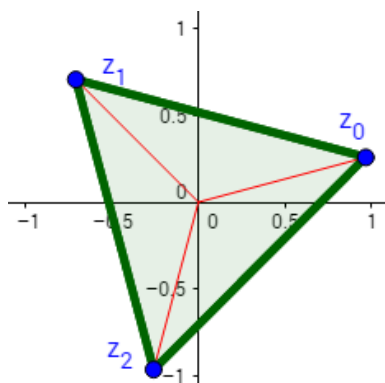


Figura 2.4: Raíz 3-esima del número complejo $z = i + 1$

2.5. Representación en forma exponencial (Euler)

El número de Euler (e) es un número constante e irracional. Se puede definir a través de la Serie de Taylor. Este número es la base para resolver ecuaciones relacionadas con el crecimiento exponencial, así como en logaritmos neperianos los cuales sirven para manejar números grandes de una manera simplificada para poder realizar cálculos.

La representación del número complejo con el número de Euler, viene dado porque Leonhard Euler aplicó la Serie de Taylor poniéndole la i del número imaginario, luego simplificó teniendo en cuenta que $i^2 = -1$. Al aplicar lo comentado, le quedó lo siguiente:

$$e^{ix} = \left(1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots\right) + i \left(x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots\right)$$

Se dio cuenta que los dos paréntesis de la expresión anterior, es la Serie de Taylor aplicada al **cos** y **sen**.

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots$$

$$\text{sen } x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Con el seno y coseno, se simplifica en:

$$e^{ix} = \cos x + i \text{sen } x$$

Este resultado es la representación del número complejo mediante el número de Euler. Además, se le conoce como la **fórmula de Euler**.

Capítulo 3

Notación Dirac

La notación de Dirac nos da una manera de expresar estados en un espacio vectorial sin atarnos a un sistema de coordenadas específico, pero que hace muy sencillo elegir una base concreta y cambiar de una base a otra.

3.1. Notación bra-ket

En un **ket** los valores están representados en un vector columna mientras que en un **bra** los valores están representados como un vector fila.

- **ket:** Representa un vector con valores complejos, se denota como: $\vec{a} = |a\rangle$
- **bra:** Representa el conjugado del vector complejo ket, se denota como: $\vec{a}^* = \langle a|$

Para **realizar la transformación** entre un **bra** y un **ket**, hay que realizar el conjugado de la transpuesta.

la base canónica en $\mathbb{C}^2$ viene representado en vector base como $e_1 = (1, 0)$ y $e_2 = (0, 1)$. Por lo que en notación ket quedaría como $|0\rangle = (1, 0)$ y $|1\rangle = (0, 1)$.

- **Producto interno:** $\langle a|b\rangle = \begin{pmatrix} a_1^* & a_2^* \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = a_1^* b_1 + a_2^* b_2$
- **Producto externo:** $\mathbf{A} = \sum_{i,j=1}^N A_{ij} |e_i\rangle\langle e_j| = \sum_{i,j=1}^N |e_i\rangle A_{ij} \langle e_j|$

Capítulo 4

¿Qué es un Qubit?

Un Quantum Bit (Qubit) es el análogo cuántico al bit clásico. Mientras el bit tiene **dos posibles estados**, el 0 y el 1, el Qubit se representa como un vector en el espacio vectorial $\mathbb{C}^2$. La base del espacio más utilizada, se denomina base computacional y se corresponde con la base canónica de $\mathbb{C}^2$, que en notación de Dirac, son los vectores $|0\rangle$ y $|1\rangle$:

$$\begin{aligned} |0\rangle &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{C}^2 \\ |1\rangle &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{aligned}$$

Un **estado cuántico** se corresponde con la descripción matemática de vector que representa al Qubit.

4.1. ¿Cómo expresarlo con notación Dirac?

Un qubit $|\Psi\rangle$ puede estar en una superposición de $|0\rangle$ y $|1\rangle$, lo que significa que es una combinación lineal entre los vectores de la base. Esto se representa como:

$$|\Psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$$

Donde α_0 y α_1 son las **amplitudes de los estados**, además son también números complejos. Por lo tanto, mientras que un bit puede solo tener 1 entre 2 valores (0 y 1), un qubit puede tener uno entre finitos posibles estados. Además, se tiene que cumplir que la norma del vector sea unitaria ($|\alpha_0|^2 + |\alpha_1|^2 = 1$).

Hay que tener en cuenta que no existe una forma directa de determinar los valores de α_1 y α_2 , ya que el estado cuántico no es directamente observable.

Antes de realizar una medición, el qubit se encuentra en una superposición de los estados $|0\rangle$ y $|1\rangle$, con ciertas probabilidades asociadas a cada uno.

Para conocer el resultado de una computación cuántica, es necesario medir los qubits. Sin embargo, este proceso de medición implica interactuar con el qubit, lo que perturba su estado y hace que este deje de estar en superposición.

Al medirlo, el qubit colapsa de forma definitiva a uno de los dos estados posibles ($|0\rangle$ o $|1\rangle$), perdiéndose la información sobre las amplitudes originales que este tenía.

La probabilidad de que el qubit colapse en un estado concreto se determina mediante la regla de Born, la cual establece que dicha probabilidad es igual al módulo al cuadrado de la amplitud correspondiente a ese estado.

La superposición, tiene distintas representaciones que son:

- **Binómica:** $x_1 + iy_1 |0\rangle + x_2 + iy_2 |1\rangle$
- **Exponencial:** $r_0 e^{i\phi_0} |0\rangle + r_1 e^{i\phi_1} |1\rangle$
- **Senos y cosenos:** $|\Psi\rangle = \cos \frac{\phi}{2} |0\rangle + e^{i\theta} \sin \frac{\phi}{2} |1\rangle$
- **Fase global:** Factor común $e^{i\gamma}$ que multiplica a todo el estado $|\psi\rangle$. Este no cambia ninguna probabilidad ni resultado físico.
- **Fase relativa:** Es la diferencia entre las amplitudes α y β . Esto influye a las inferencias y los resultados cuando se miden en otras bases (X, Y) .

4.2. Esfera de Bloch

La esfera de Bloch es una representación visual del estado de un qubit. Cada punto sobre su superficie corresponde a una posible superposición de los estados base $|0\rangle$ y $|1\rangle$. Los polos superior e inferior representan los estados $|0\rangle$ y $|1\rangle$, respectivamente, y el estado del qubit se indica con una flecha que apunta hacia el punto que lo describe.

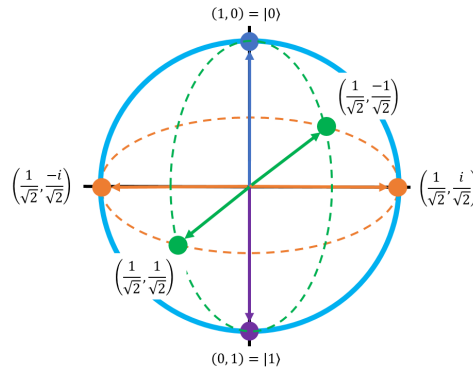


Figura 4.1: Imagen obtenida de <https://stem.mitre.org/quantum/quantum-concepts/bloch-sphere.html>

Importante: la notación $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ se corresponde con el vector estado $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$ que a su vez significa:

$$\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Los seis puntos principales de la esfera de Bloch, son los que se ve en la Figura 4.1. A estas coordenadas principales, se le pueden asignar etiquetas.

- **Base de Hadamard:** También se le conoce como eje X de Pauli. El punto $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ se corresponde con $|+\rangle$. A su vez, el punto opuesto $\left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right)$ se corresponde con $|-\rangle$
- **Base circular:** También se le conoce como eje Y de Pauli. El punto $\left(\frac{1}{\sqrt{2}}, \frac{i}{\sqrt{2}}\right)$ se corresponde con $|+i\rangle$. A su vez, el punto opuesto $\left(\frac{1}{\sqrt{2}}, -\frac{i}{\sqrt{2}}\right)$ se corresponde con $| -i\rangle$

Con dicha asignación de etiquetas, queda la siguiente esfera:

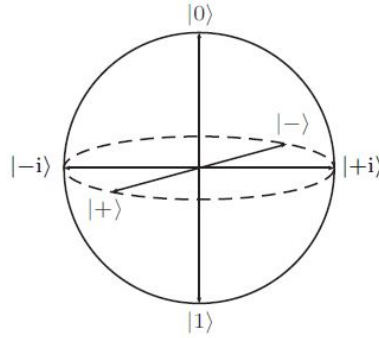


Figura 4.2: Imagen obtenida de https://en.wikipedia.org/wiki/Bloch_sphere

4.3. Puertas cuánticas

Al igual que para manipular los valores de los bits existen puertas lógicas clásicas como OR, AND, NOT, etc. Los ordenadores cuánticos, manipulan los valores de los qubits usando puertas cuánticas. Es decir, cualquier algoritmo cuántico, se tiene que implementar usando puertas cuánticas.

Las puertas cuánticas, se **representan** mediante **matrices unitarias** U . Esto significa que conservan la probabilidad total de todos los posibles resultados (gracias a las propiedades de matrices unitarias), en otras palabras, no se pierde la información durante la operación.

Al aplicar una puerta cuántica a un estado cuántico, realizas $|\psi'\rangle = U|\psi\rangle$. Extendiendo dicha operación, se obtiene $|\psi'\rangle = \alpha'_0|0\rangle + \alpha'_1|1\rangle$. Aunque se aplique alguna puerta cuántica, se sigue manteniendo la ecuación $|\alpha'_0|^2 + |\alpha'_1|^2 = 1$.

Las puertas cuánticas para un único qubit son:

- **Pauli I:** Se le considera una extensión del conjunto de matrices de Pauli. La puerta viene denotada por la matriz identidad I .

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **Pauli X:** Se conoce también como la puerta NOT. La representación matricial de dicha puerta es:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Cuando se aplica la puerta de Pauli X, el resultado es que el $|0\rangle$ y $|1\rangle$ se invierten.

- **Pauli Y:** La representación matricial de esta puerta es:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Si un qubit en superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ entra en la puerta Y, el resultado es:

$$Y |\psi\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} -i\alpha_2 \\ i\alpha_1 \end{bmatrix} = -i\alpha_2 |0\rangle + i\alpha_1 |1\rangle$$

- **Pauli Z:** La representación matricial es:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Cuando la puerta Pauli Z opera sobre un qubit, la fase cambia. Matemáticamente al entrar un qubit en estado de superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ por la puerta Z, el resultado es el siguiente:

$$Z |\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} \alpha_1 \\ -\alpha_2 \end{bmatrix} = \alpha_1 |0\rangle - \alpha_2 |1\rangle$$

- **Puerta Hadamard (H):** Actúa sobre un qubit definido, al actuar, produce un estado de superposición como salida. La forma matricial es:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Si un qubit en estado de superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ atraviesa esta puerta el resultado es:

$$H |\psi\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} \alpha_1 + \alpha_2 \\ \alpha_1 - \alpha_2 \end{bmatrix} = \left(\frac{\alpha_1 + \alpha_2}{\sqrt{2}} \right) |0\rangle + \left(\frac{\alpha_1 - \alpha_2}{\sqrt{2}} \right) |1\rangle$$

Capítulo 5

Procesamiento Lenguaje Natural (PLN)

5.1. ¿Qué es el procesamiento de lenguaje natural?

El PLN, es la rama de la IA encargada de la interacción entre computadoras y el lenguaje humano. Este campo integra la lingüística computacional, basada en el modelado de reglas del lenguaje, con modelos estadísticos y algoritmos de aprendizaje profundo (Deep Learning (DL)) y aprendizaje automático (ML).

El PLN forma parte a día de hoy de muchos motores de búsqueda, chatbots para atención al cliente (ya sea por escrito o por voz), y asistentes digitales como Alexa de Amazon o Siri de Apple.

El campo de la computación lingüística incluye tradicionalmente diversas fases para el análisis. Para el análisis de las palabras existen dos formas principales de hacerlo:

- **Análisis de dependencias:** Examina las relaciones gramaticales binarias entre las palabras, identificando, por ejemplo, qué palabra modifica a cuál o cuál es el sujeto y el verbo principal.
- **Análisis de constituyentes:** Construye un árbol sintáctico que representa la estructura gramatical anidada de una oración, descomponiéndola en sintagmas o frases ordenadas jerárquicamente.

5.2. Fases del PLN

Las fases para realizar el PLN se pueden observar en la siguiente imagen: A continuación, se detalla en qué consiste cada una de ellas:

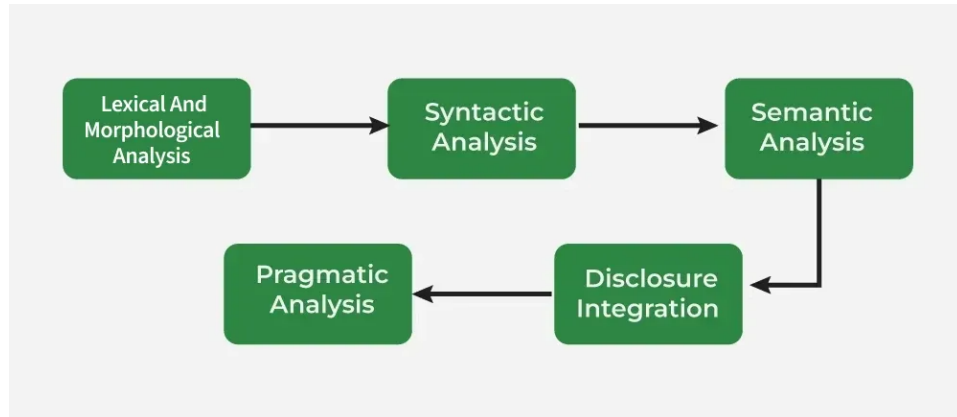


Figura 5.1: Imagen obtenida de <https://www.geeksforgeeks.org/machine-learning/phases-of-natural-language-processing-nlp/>

5.2.1. Preprocesamiento y Tokenización

Esta fase constituye el primer nivel de procesamiento y su objetivo es transformar el texto crudo en una secuencia de unidades discretas manejables, integrando tareas de análisis léxico y morfológico.

1. **Preprocesamiento:** Incluye la limpieza del texto, normalización (convertir a minúsculas), eliminación de caracteres especiales, y en ocasiones, la eliminación de *stopwords* (palabras muy frecuentes sin gran carga semántica como ".el", "de", "z"). También incluye técnicas de análisis morfológico como:

- *Lematización:* Convertir palabras a su forma base (e.g., "soy" → "ser").
- *Stemming:* Reducir palabras a su raíz (e.g., "niñero" → "niño").

2. **Tokenización:** Es el proceso crítico de segmentar el flujo de texto en unidades atómicas significativas o *tokens*.

Ejemplo: La oración *Me gusta programar* se transformaría en la lista de tokens: ["Me", "gusta", "programar"].

Dependiendo del modelo, los tokens pueden ser palabras completas, subpalabras (como en los modelos Generative Pre-trained Transformer (GPT)) o incluso caracteres. Una vez tokenizado el texto, cada token se asocia a un índice numérico único en un vocabulario, preparando el terreno para la generación de *Embeddings*.

3. **Etiquetado gramatical (Part-of-Speech (POS) Tagging):** Asigna a cada token una categoría morfológica (sustantivo, verbo, adjetivo,

etc.) basándose en su definición y contexto. Esto permite desambiguar el rol sintáctico de cada palabra.

Ejemplo: Para los tokens del ejemplo anterior, el etiquetado resultante sería: ["Me" → Pronombre, "gusta" → Verbo, "programar" → Verbo].

5.2.2. Análisis sintáctico

El análisis sintáctico examina la organización de las palabras dentro de una oración para verificar que se cumple con las reglas gramaticales del idioma. El objetivo fundamental de esta fase es la generación de un **árbol de análisis sintáctico**, el cual representa jerárquicamente la estructura gramatical del texto analizado. Esta representación estructurada permite a los sistemas computacionales interpretar las relaciones de dependencia y jerarquía entre los distintos elementos léxicos. Las tareas principales que abarca son:

- **Etiquetado gramatical:** Corresponde al mismo proceso descrito anteriormente, validando la estructura en este nivel.
- **Resolución de ambigüedad sintáctica:** Se ocupa de determinar el significado correcto de una palabra basándose en su contexto gramatical. Por ejemplo, distinguir la acepción de la palabra *banco* en *Fui a sacar dinero al banco* (entidad financiera) frente a la frase *Me senté en el banco del parque* (sitio para sentarse y descansar).

5.2.3. Análisis semántico

Esta fase se centra en la coherencia lógica y la relevancia contextual del enunciado. Su objetivo principal es interpretar el significado de las palabras, analizando tanto su definición de diccionario como su variación según el contexto de uso. De este modo, se busca comprender el rol semántico de cada palabra para construir una representación lógica del mensaje. Las tareas fundamentales que abarca esta fase son:

1. **Named Entity Recognition (NER) (NER):** Se ocupa de la identificación y clasificación de elementos clave en el texto, tales como nombres de personas, ubicaciones geográficas, organizaciones, etc.
Ejemplo: En la oración *Mercedes anunció un nuevo coche en Berlín*, el sistema NER identificaría a *Mercedes* como una *Organización* y a *Berlín* como un *Lugar*.
2. **Word Sense Disambiguation (WSD) (WSD):** Identifica el significado correcto de las palabras en función del contexto.
Ejemplo: El término *Banco* puede interpretarse como una *entidad financiera* o como *un lugar donde sentarse* dependiendo del resto de palabras de la oración.

5.2.4. Integración del discurso

Este proceso analiza cómo las oraciones individuales se conectan y relacionan entre sí para entender el contexto global. El objetivo de esta fase es garantizar que el significado del texto mantenga su coherencia lógica a través de los distintos párrafos y frases. Los aspectos fundamentales de esta fase son:

- **Resolución de anáforas:** Consiste en identificar la conexión entre un pronombre y lo mencionado previamente en el texto.
Ejemplo: En la secuencia *Esther fue a comprar. Ella compró fruta*, el sistema debe vincular el pronombre *Ella* con *Esther*.
- **Referencias contextuales:** Aborda la interpretación de palabras o expresiones cuyo significado completo depende intrínsecamente de la información proporcionada en oraciones anteriores o posteriores.
Ejemplo: La frase *Fue un buen día* adquiere un significado concreto únicamente si se analiza dentro del contexto de la conversación.

5.2.5. Análisis pragmático

Esta fase final analiza más allá de la interpretación literal de las oraciones para inferir el significado real que se pretende transmitir. A diferencia del análisis semántico, que se ocupa del significado proposicional de las palabras, el análisis pragmático examina factores como el tono y el contexto. Las tareas principales de esta fase son:

- **Detección de intenciones:** El sistema debe ver el propósito de la comunicación (si es una petición, una queja, una orden, etc.).
Ejemplo: Ante la pregunta *¿Tienes hora?*, el análisis pragmático entiende que la intención real del usuario es saber qué hora es, no una pregunta de sí o no.
- **Interpretación de lenguaje figurado:** Se encarga de identificar metáforas, ironías o frases hechas.
Ejemplo: En la expresión *Echar una mano*, el análisis pragmático entiende que significa *ayudar a alguien*.

Capítulo 6

Procesamiento del Lenguaje Natural Cuántico (PLNC)

El procesamiento de datos clásicos utilizando algoritmos de aprendizaje automático a través de sistemas cuánticos, ha generado una línea de investigación de especial foto de interés estos últimos años. El QML se ha utilizado con distintos propósitos: profundizar en el uso de los fenómenos cuánticos para sistemas de aprendizaje, explorar la capacidad de los computadores cuánticos en aprender sobre datos cuánticos, y la posibilidad de reformular e implementar algoritmos de ML en hardware cuántico.

Como ocurrió anteriormente en el caso del ML clásico, el rápido crecimiento de los algoritmos de QML, tanto desde el lado del hardware como del software (en términos de dispositivos y algoritmos), ha involucrado al campo de PLN. Esto ha dado lugar al llamado PLNC, definido como la implementación del procesamiento del lenguaje natural en hardware cuántico.

6.1. Motivación para el uso de PLNC

La investigación del Procesamiento de Lenguaje Natural Cuántico (PLNC) está motivada fundamentalmente por las limitaciones físicas y computacionales de los modelos clásicos ante la creciente complejidad del lenguaje, especialmente tras la aparición de los LLMs. Los sistemas tradicionales requieren recursos masivos de tiempo y memoria para procesar grandes conjuntos de datos (corpus, como por ejemplo la Wikipedia). Además, los modelos PLN clásicos sufren degradación en precisión y fiabilidad al enfrentarse por ejemplo a la polisemia, es decir, cuando una palabra tiene distintos significados según el contexto, debido a las restricciones de las arquitecturas clásicas (frecuentemente basadas en estructuras de árbol) [15].

Para afrontar estos retos, el PLNC aprovecha fenómenos de la mecánica cuántica como la **superposición** y el **entrelazamiento**. Al operar en el espacio de *Hilbert*, esta arquitectura permite almacenar múltiples significados

y contextos simultáneamente en un solo registro y ejecutar operaciones en paralelo. Esto no solo ofrece una ventaja exponencial acelerando la computación [39], sino que también proporciona una 'expresividad' superior, capturando las dependencias semánticas del lenguaje humano de una manera que los espacios numéricos clásicos replican con menor eficiencia.

No obstante, hay que tener en cuenta que, aunque teóricamente el PLNC puede resolver estos problemas, aún no se ha podido realizar una comparación a gran escala en hardware real (sin usar simuladores) frente a la computación clásica. Los modelos cuánticos actuales se han implementado sobre corpus pequeños, por lo que todavía no representan la escala completa que maneja un LLM moderno.

6.2. Codificación de información clásica en estados cuánticos

Una de las hipótesis fundamentales que motiva la intersección entre el procesamiento de lenguaje natural y la computación cuántica es la existencia de una relación estructural directa entre las características lingüísticas (sintáctica y semántica) y los estados cuánticos.

El framework Distributional Compositional Categorical (DisCoCat) [43] es el más conocido en formalizar esta relación. Nace de proponer la unificación de los dos enfoques clásicos del significado:

- **Distribucional:** Basado en la estadística y el contexto, que es la base de los vectores de palabras modernos.
- **Composicional:** Basado en la estructura gramatical, donde el significado de una frase depende de sus partes y de cómo se combinan (reglas sintácticas).

La relevancia de DisCoCat para el PLNC radica en que modela el lenguaje utilizando diagramas de palabras y productos tensoriales, una estructura matemática que es a la que describe a su vez la computación cuántica. Esto sugiere que los ordenadores cuánticos son, de manera nativa, idóneos para procesar el lenguaje natural.

En la imagen, las cajas representan el significado de las palabras que se transmiten mediante los cables", esto es similar a la técnica de Dependency Parse Tree (DPT) muy popular en la literatura lingüística. En la frase, el sujeto es "Dani", mientras que "pizza.^{es} el objeto, ambas palabras se relacionan mediante el verbo *comez* la combinación de estas palabras, crean el significado conjunto de la frase. Con DisCoCat, el significado de la oración se construye utilizando una gramática preagrupada mediante composición de productos tensoriales. Es decir, quedaría representado de la siguiente manera:

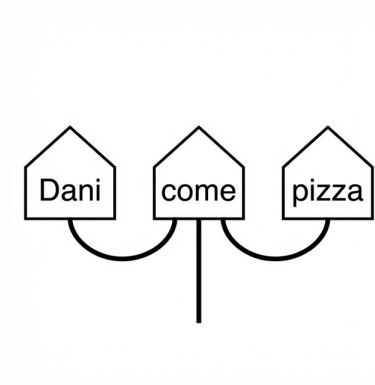


Figura 6.1: Diagrama de la oración "Dani come pizza."^{en} el formalismo DisCoCat.

$$(\overrightarrow{\text{Dani}} \otimes \overrightarrow{\text{subj}}) \otimes \overrightarrow{\text{come}} \otimes (\overrightarrow{\text{pizza}} \otimes \overrightarrow{\text{obj}})$$

Este vector generado en el espacio de productos tensoriales, se puede considerar como el significado de la oración "Dani come pizza". Por ende, se dice que este framework lo ha expresado en términos de computación cuántica.

Cabe destacar que, aunque el modelo DisCoCat original funciona perfectamente sin referencias explícitas a la teoría cuántica (a pesar de originarse en el formalismo de la Categorical Quantum Mechanics (CQM)). La novedad yace en el argumento de que el PLNC puede considerarse "nativo cuántico".

Esto se debe a que la teoría cuántica y el lenguaje natural comparten una misma estructura de interacción y el uso fundamental de espacios vectoriales para describir sus estados. Esta estructura de interacción determina por completo la estructura los procesos, incluyendo la especificación de los espacios donde "viven" los estados. Esta coincidencia estructural implica que el lenguaje natural podría encajar de manera más natural y eficiente en hardware cuántico que en hardware clásico.

6.2.1. Técnicas de codificación de datos clásicos a estados cuánticos

Al igual que en computación clásica la información necesita ser codificada de manera que los modelos pueda entender los datos que se le proporciona y así trabajar con ellos, la computación cuántica necesita poder codificar los datos de la computación clásica (texto en este caso, ya que estamos dentro del ámbito de PLN y PLNC), a estados cuánticos. Para ello existe diversas técnicas que son:

- **Basis Encoding:** Utiliza n qubits. Complejidad $O(1)$. Simple, entradas cortas y simbólicas.
- **Angle Encoding:** Utiliza n qubits. Complejidad $O(n)$. Resiliente al ruido, vectores de características pequeños.
- **Amplitude Encoding:** Utiliza $\log_2(n)$ qubits. Complejidad $O(2^n)$. Eficiente en memoria, vectores de frases completas.
- **Hamiltonian Encoding:** Utiliza $\log_2(n)$ qubits. Complejidad $O(t)$. Estructura de grafos, análisis sintáctico (parsing), traducción.

De todas estas técnicas, se ha optado por implementar la técnica de *Basis Encoding* [30]. Este tipo de codificación se utiliza principalmente cuando un número real debe ser matemáticamente manipulado en un algoritmo cuántico. Su funcionamiento consiste en representar un número real como un número binario y luego transformarlo en un estado cuántico en la base computacional.

El estado cuántico embebido es la traducción bit a bit de una cadena binaria a los estados correspondientes de los subsistemas cuánticos. Por lo tanto, un bit de información clásica está representado por un subsistema cuántico. Actuar sobre características binarias codificadas como qubits otorga mayor flexibilidad computacional para diseñar algoritmos cuánticos.

Supongamos que queremos codificar la frase "María estudia medicina" utilizando *Basis Encoding*. Primero, definimos un vocabulario simplificado donde asignamos un índice entero único a cada palabra y fijamos una longitud de $\tau = 5$ bits por palabra (lo que permitiría un vocabulario de hasta $2^5 = 32$ palabras).

- *María* $\rightarrow$ Índice 3 $\rightarrow$ 00011
- *estudia* $\rightarrow$ Índice 12 $\rightarrow$ 01100
- *medicina* $\rightarrow$ Índice 21 $\rightarrow$ 10101

El vector de características de la frase corresponde entonces a la concatenación de estas secuencias binarias: $b = 00011 \ 01100 \ 10101$.

Finalmente, esta secuencia se representa mediante el estado cuántico conjunto en la base computacional:

$$|\psi\rangle_{\text{frase}} = |00011\rangle \otimes |01100\rangle \otimes |10101\rangle = |00011 \ 01100 \ 10101\rangle$$

De esta forma, hemos codificado una frase de lengua clásica directamente en un estado cuántico en la base computacional.

6.3. Circuitos Variacionales Cuánticos (CVC) como modelos de aprendizaje

En 2016, se tuvo acceso al primer ordenador cuántico en la nube aunque el ruido y las limitaciones de qubits de las que se disponía, hacía que fuera difícil implementar algoritmos cuánticos.

A pesar de ello, hubo una emoción muy grande con lo que se podría hacer con estos nuevos ordenadores llamados Noisy Intermediate-Scale Quantum (NISQ). Estos nuevos ordenadores tenían entre 50 y 100 Qubits, lo que les permitía sobrepasar en rendimiento a los mejores superordenadores clásicos en algunas áreas matemáticas [2, 46].

Los Circuitos Variacionales Cuánticos (CVC) nacieron como la mejor estrategia para hacer frente a los ordenadores NISQ. Debido a las limitaciones de estos dispositivos, se utilizó la estrategia de optimización basado en el aprendizaje. No son más que una analogía cuántica a los técnicas existentes del campo de ML como pueden ser las redes neuronales.

En definitiva, son algoritmos cuánticos que tienen parámetros libres [25]. Al igual que cualquier otro circuito cuántico estándar, se necesita hacer uso de estos tres conceptos básicos:

1. Preparación con un **estado inicial** fijo.
2. Un **circuito cuántico** $U(\theta)$ parametrizado con un conjunto de parámetros libres θ .
3. **Medición** de un observable $\hat{B}$ en la salida. Este observable puede estar compuesto por varios observables cada uno de manera local a un cable del circuito, o bien, a un conjunto de cables del circuito.

Normalmente, los valores esperados $f(\theta) = \langle 0|U^\dagger(\theta)\hat{B}U(\theta)|0\rangle$ de uno o varios circuitos, definen un coste escalar para la tarea específica. Los parámetros libres $\theta = (\theta_1, \theta_2, \dots)$ son ajustados para optimizar esta función de coste.

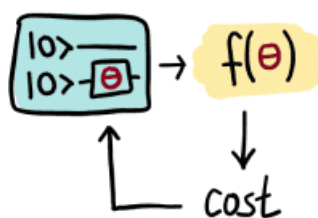


Figura 6.2: Coste de un Circuito Cuántico Variacional, imagen obtenida de: [25]

Los circuitos cuánticos variacionales son entrenados mediante un algoritmo de optimización clásico que realiza consultas al hardware cuántico. La optimización suele ser un esquema iterativo que busca mejores candidatos para los parámetros en cada paso.

Los parámetros variacionales θ , posiblemente junto con un conjunto adicional de parámetros no adaptables $x = (x_1, x_2, \dots)$, entran en el circuito cuántico como argumentos para las puertas del circuito. Esto nos permite convertir *información clásica* (los valores θ y x) en *información cuántica* (el estado cuántico $U(x; \theta)|0\rangle$). Como veremos en el ejemplo a continuación, los parámetros de puerta no adaptables suelen desempeñar el papel de *entradas de datos* en el aprendizaje automático cuántico.

La información cuántica se convierte de nuevo en información clásica evaluando el valor esperado del observable $\hat{B}$,

$$f(x; \theta) = \langle \hat{B} \rangle = \langle 0 | U^\dagger(x; \theta) \hat{B} U(x; \theta) | 0 \rangle.$$

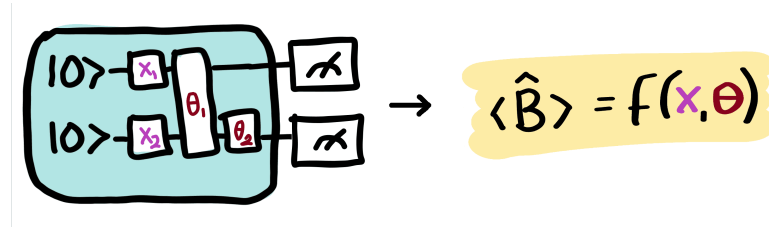


Figura 6.3: Circuito variacional cuántico con parámetros modificables x y no modificables θ . Imagen obtenida de: [25]

Capítulo 7

Representación Vectorial de Palabras (Word Embeddings)

Los word embeddings son una de las técnicas más usadas en Natural Language Processing (NLP). Gracias a los word embeddings, existen lenguajes de modelo como GPT-3, GPT-4, etc.

Estos algoritmos son rápidos y pueden generar secuencias de lenguaje y realizar otras tareas derivadas con alta precisión, incluyendo la comprensión contextual, propiedades semánticas y sintácticas, así como la relación lineal entre las palabras.

En esencia, estos modelos utilizan los embeddings como un método para extraer patrones de secuencias de texto o voz. Pero, ¿cómo lo hacen?

En las siguientes secciones se explorará en mayor profundidad los word embeddings así como conceptos más avanzados de estos.

7.1. Introducción al concepto de *Embedding*

De manera intuitiva, un *embedding* de texto es una representación matemática que proyecta un fragmento de texto en un espacio latente de alta dimensionalidad. En este espacio, la posición del texto se define mediante un vector, donde cada componente del vector corresponde a una dimensión. Al igual que en un plano cartesiano se tienen dos dimensiones, en un embedding, se suele trabajar con muchas más dimensiones para representar una palabra en el espacio[16]. Por ejemplo, el modelo de **OpenAI** *text-embedding-ada-002* el cual tiene un espacio de dimensión de 1536. Es decir, cada vector está formado por 1536 componentes.

Un ejemplo de cómo se verían los valores que toma cada dimensión es el siguiente:

“A text embedding is a piece of text projected into a high-dimensional latent space. The position of our text in this space is a vector, a long sequence of numbers. Think of the two-dimensional cartesian coordinates from algebra class, but with more dimensions—often 768 or 1536.” [16]

Al procesar el párrafo anterior mediante el modelo *text-embedding-ada-002* mencionado, se obtiene la gráfica de la Figura 7.1, donde cada barra vertical representa el valor asignado a una de sus 1536 dimensiones.

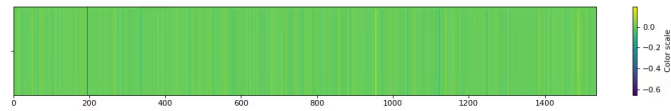


Figura 7.1: Representación gráfica de los valores en las 1536 dimensiones de un embedding generado por el modelo *text-embedding-ada-002*. Fuente: [16].

Un **espacio de embedding** o **espacio latente** se define matemáticamente como un espacio donde los elementos están posicionados de manera más cercana uno del otro cuando tienen valores parecidos, mientras que si difieren dichos elementos, se posicionan más lejos entre ellos. En el caso del PLN, frases que son semánticamente más similares a otras, tendrán un vector de embedding similar por lo que estarán más juntas en el espacio.

Aunque la comparación entre frases es una de las aplicaciones más conocidas, los *embeddings* también se aplica en modelos de ML y en arquitecturas de redes neuronales complejas.

7.1.1. Embeddings de palabras (*Word Embeddings*)

Mientras que el concepto general de *embedding* puede aplicarse a frases completas (como en el ejemplo introducido), su uso más extendido e histórico se encuentra en su aplicación a nivel atómico: las palabras. Un *embedding* de palabras, es un vector de longitud fija el cual su único cometido, es representar una única palabra o *token* [1].

El diseño y creación de estos vectores se clasifica principalmente en dos grandes familias estratégicas, dependientes de sus algoritmos de generación: los **modelos basados en predicciones** y los **modelos basados en conteos**.

Modelos basados en predicciones

Esta técnica de representación surgió fuertemente ligada al desarrollo de los MNLs. A diferencia de los enfoques estadísticos tradicionales de recuento,

un modelo predictivo se centra en aprender el significado de un término de forma prospectiva, es decir, “adivinando”.

Para lograrlo, emplean una red neuronal sencilla dotada de una ventana de contexto de tamaño fijo y móvil (por ejemplo, avanzando de cinco en cinco palabras). A medida que lee el texto, el modelo intenta predecir el significado de una palabra central en base a las palabras vecinas de esta ventana, o a la inversa, predecir el contexto apoyándose únicamente en dicho término central.

El aspecto fundamental en todo esto, es que tras finalizar el entrenamiento, el objetivo de ML deja de ser la capacidad del modelo para adivinar con exactitud. En su lugar, de la red neuronal, se extraen los **pesos y valores** que ha ajustado internamente. Estos valores que fueron ajustados durante el entrenamiento, se convierten directamente en el *word embedding*. En consecuencia, el mecanismo iterativo garantiza que aquellas palabras empleadas en contextos idénticos acaben reteniendo representaciones vectoriales prácticamente iguales.

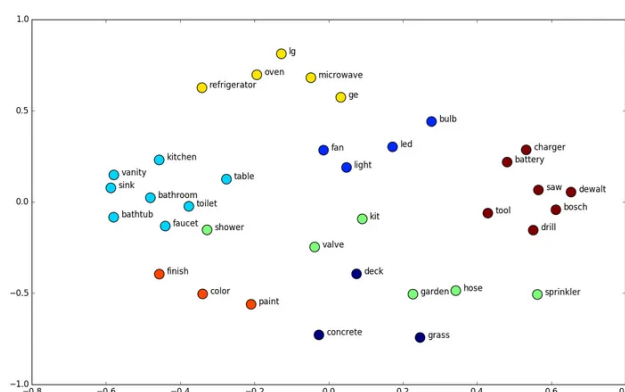


Figura 7.2: Representación visual de un modelo basado en predicciones para generar embeddings de palabras. Fuente: <https://neptune.ai/blog/word-embeddings-guide>.

Una de las arquitecturas más representativas dentro de la predicción de *embeddings* es la famosa familia algorítmica **Word2Vec**, la cual se detalla de forma exhaustiva en el próximo capítulo.

Modelos basados en conteos

Por su parte, la segunda categoría de modelos hace uso de la estadística global a lo largo de un documento completo (*corpus*). Estos modelos se aprovechan de los recuentos de coocurrencia (palabra y contexto) estructurando toda la información como grandes *matrices de palabra-contexto*.

El funcionamiento radica en la creación de una matriz gigantesca donde se contabiliza el número de apariciones conjuntas en las que una palabra acompaña a todo un posible contexto del propio texto. No obstante, si el conjunto de datos cuenta con n palabras distintas, las dimensiones de la matriz ascenderán a un total de $n \times n$. Esto acarrea severamente un cuello de botella de almacenamiento. Adicionalmente, la inmensa mayoría de cruces de dicha matriz representarán palabras que jamás han coexistido en el texto (por ejemplo, .°vejaz "pantalla"), llenando de ceros el modelo de datos. Esta estructura tan ineficiente es conocida como **matriz dispersa**.

Para solventar esta carencia y la falta de memoria, se reduce la dimensionalidad a través de complejas operaciones de álgebra lineal, como puede ser la Descomposición en Valores Singulares (DVS). Con esto logran sintetizar enormes matrices de ceros, transformándolos en **vectores cortos y densos**. De tal modo, que los conceptos globalmente equivalentes retengan valores numéricamente similares sin desperdiciar memoria.

La **mayor ventaja** de esta táctica estadística contable, frente al sistema predictivo, reside precisamente en que es capaz de aprovechar la información global de todo el *corpus*, un aspecto estadístico que se le escapa a las ventanas de lectura en las redes neuronales por definición.

El modelo híbrido con más popularidad en este ámbito es el de **GloVe (Global Vectors)**. Desarrollado por un equipo de investigadores de la Universidad de Stanford, logra exitosamente agrupar los beneficios numéricos de las matrices grandes de conteo global, con las altas capacidades de optimización paramétrica propias de los modelos del ML.

7.2. Mecanismos geométricos de los embeddings

Una vez comprendido que un embedding proyecta palabras en un espacio continuo donde la cercanía espacial equivale a similitud semántica, surgen dos interrogantes fundamentales en el diseño de estos sistemas: ¿cómo se estructuran los ejes de este espacio para encapsular el significado complejo del lenguaje? y ¿cómo se calcula con precisión matemática la distancia entre dos vectores?

Para responder a la primera cuestión, es necesario analizar el papel de la **información latente** [16], la cual permite pasar de representaciones dispersas a vectores densos basados en atributos ocultos. Para la segunda, se debe explorar **métricas de distancia** específicas que sean efectivas en espacios de tantas dimensiones.

7.2.1. Métricas de Distancia

Como se explicó con anterioridad, una de las características principales de un embedding representado en el espacio, es que mantiene la distancia. Los vectores de alta dimensionalidad que se utilizan en los embeddings de

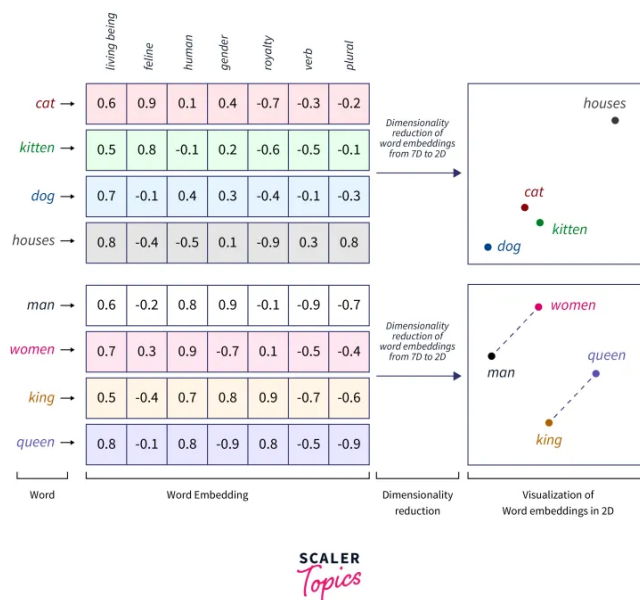


Figura 7.3: Representación visual de un modelo basado en conteos para generar embeddings de palabras. Fuente: <https://www.scaler.com/topics/tensorflow/tensorflow-word-embeddings/>.

palabra (así como en los LLMs), no son intuitivos. Pero la intuición espacial se mantiene igual mientras reducimos la escala de estos.

Un ejemplo básico es el siguiente: imagina un plano de planta bidimensional de una biblioteca de una sola planta. Los usuarios de la biblioteca son amantes de los gatos, de los perros, o se sitúan en algún punto intermedio. El objetivo es colocar los libros sobre gatos cerca de otros libros de gatos, y los libros sobre perros cerca de otros libros de perros.

El enfoque más sencillo se conoce como el modelo de bolsa de palabras. Consiste en establecer un eje de perros a lo largo de una pared y un eje de gatos perpendicular a él. A continuación, se cuentan las ocurrencias de las palabras "gato" "perro" en cada libro y se ubica dicho libro en su punto correspondiente dentro del sistema de coordenadas ($\text{perro}_x, \text{gato}_y$).

Pensemos ahora en un sistema de recomendación sencillo. A partir de la selección anterior de un libro, ¿qué se podría sugerir a continuación? Bajo la suposición (muy simplificada) de que estas dos dimensiones capturan adecuadamente las preferencias del lector, basta con buscar el libro que se encuentre más cerca. En este caso, el sentido intuitivo de cercanía es la distancia euclídea —el camino más corto entre dos libros—:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Siguiendo el razonamiento lógico, si se analiza a través de la distancia euclídea, se pondrá el libro (perro₅, gato₁) más cerca del libro (perro₁, gato₅) que de un libro mucho más extenso o enciclopédico como (perro₁₀₀, gato₁). Esto plantea un grave problema semántico: un lector de libros cortos de perros con proporción (5, 1) seguramente prefiera a continuación una enciclopedia gigante sobre perros de (100, 1) antes que un libro corto sobre gatos de (1, 5). La métrica euclídea penaliza severamente la longitud en palabras del libro (magnitud) omitiendo por completo el reparto del tema de este (la proporción).

Cuando resulta más relevante concentrarse en los pesos relativos temáticos que en sus magnitudes absolutas, los vectores se normalizan. Esto se logra dividiendo en ambos la cantidad de apariciones de perros y gatos para obtener la conocida **similitud coseno** [28]. De manera más visual, este cálculo equivale geométricamente a proyectar todos los puntos disponibles en el espacio sobre una circunferencia de radio unidad (es decir, que el módulo de todos los vectores es 1) y medir la distancia a lo largo del arco que los une. Al ignorar la longitud del documento original, se evalúa únicamente la dirección del vector respecto al origen, logrando una métrica de similitud mucho más robusta y natural al lenguaje humano.

Matemáticamente, esta métrica se define mediante el cociente del producto escalar de dos determinados vectores entre el producto de sus magnitudes:

$$\text{similitud}(\mathbf{p}, \mathbf{q}) = \cos(\theta) = \frac{\mathbf{p} \cdot \mathbf{q}}{\|\mathbf{p}\| \|\mathbf{q}\|} = \frac{\sum_{i=1}^n p_i q_i}{\sqrt{\sum_{i=1}^n p_i^2} \sqrt{\sum_{i=1}^n q_i^2}}$$

Un pensamiento intuitivo para entender la fórmula de similitud coseno, es que si dos vectores apuntan en la misma dirección, el ángulo entre ellos es 0 por lo que el coseno es 1. Si son ortogonales, significa que no comparten una 'direccionalidad', el coseno es 0.

En el contexto que nos encontramos de embeddings de palabras, hay que pensar que cada vector actúa como una flecha que apunta desde el origen hasta la posición de la palabra en nuestra librería imaginada. Palabras que son similar semánticamente, tendrán vectores que apuntan en direcciones similares, es decir, la similitud de coseno será mayor.

7.2.2. Información Latente

Cualquier texto suele emplear sinónimos o términos relacionados para referirse a un mismo concepto; por ejemplo, volviendo al mismo ejemplo de an-

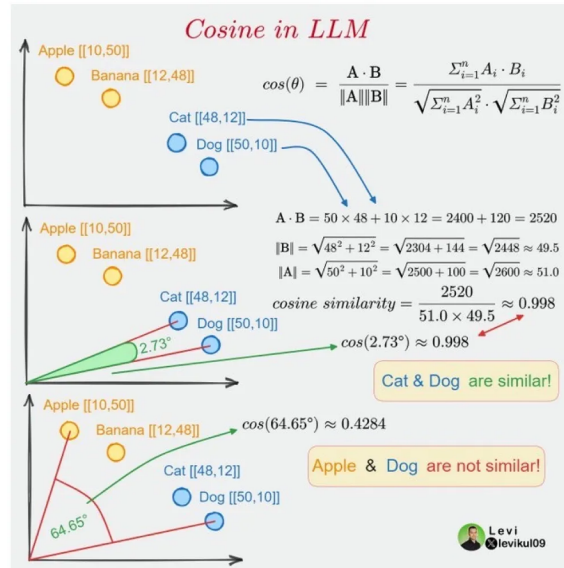


Figura 7.4: Ejemplo visual de la representación de vectores y su cercanía empleando la similitud coseno. Fuente: <https://x.com/levikul09>.

tes, un libro sobre perros utilizará frecuentemente palabras como canino”. En un modelo básico de bolsa de palabras, incorporar este nuevo vocablo requiere añadir una dimensión perpendicular adicional a las ya existentes, transformando nuestro anterior sistema de coordenadas en $(\text{perro}_x, \text{gato}_y, \text{canino}_z)$.

Bajo este mismo principio, resulta muy sencillo continuar agregando nuevas dimensiones para representar más palabras, consiguiendo un espacio multidimensional $(\text{perro}_x, \text{gato}_y, \text{canino}_z, \text{felino}_w)$. Sin embargo, este enfoque tan básico conlleva dos grandes problemas que imposibilitan su aplicación a gran escala: rompe de inmediato la interpretabilidad espacial (como lo era la sencilla idea inicial de una biblioteca bidimensional) y provoca una explosión en los requisitos necesarios para el cómputo.

Resulta evidente que, a nivel computacional, esta aproximación es extremadamente ineficiente a medida que aumenta el tamaño del vocabulario. El español, por ejemplo, cuenta con más de 93.000 palabras según el Diccionario de la Real Academia Española [8]. Manejar vectores con decenas de miles de dimensiones implica calcular una ingente cantidad de combinatorias. Además, se agrava el problema de la dispersión de datos: en un documento dado, la gran mayoría del vocabulario total no aparecerá, rellenando los vectores con innumerables ceros que penalizan drásticamente el rendimiento de los modelos estadísticos y de aprendizaje automático.

La solución a la alta dimensionalidad radica en abstraer el conteo literal hacia ejes que representen verdaderos atributos semánticos. Si el modelo es capaz de identificar la relación semántica entre “gatoz” “felino”, podría

36 7.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert

agrupar la contribución de ambas en una única variable latente compartida. Esto no solo comprime el tamaño de los vectores, haciendo a los algoritmos mucho más eficientes y veloces, sino que también genera una representación geométrica mucho más afín a la intuición humana.

De la misma forma, términos más amplios como "mascota" o "mamífero" representan características semánticas compartidas simultáneamente por "perro" y "gato". En un espacio latente bien optimizado, estas palabras genéricas actúan como atributos comunes que vinculan y aproximan ambos animales en ciertas dimensiones del espacio. Por otro lado, si la elevada reducción dimensional provocara una superposición excesiva donde términos que debieran ser distintos terminen colapsando —por ejemplo, confundándose ambos en el mismo punto exacto—, esta carencia de precisión podría mitigarse introduciendo nuevos ejes latentes. Estos nuevos atributos ocultos de diferenciación (por ejemplo, proyectando qué palabras se relacionan estrechamente con "mesa" frente a los animales vivos mencionados anteriormente) permiten desenredar los agrupamientos de datos semánticos complejos.

7.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert

Como se vio en capítulos anteriores, los estados en computación cuántica están representados por vectores definidos en un espacio complejo, el cual se conoce formalmente como **espacio de Hilbert** ($\mathcal{H}$). Por su parte, en el PLN clásico se modela el significado de las palabras mapeándolas a vectores en un espacio vectorial puramente real ($\mathbb{R}^n$).

A partir de esta analogía natural, surge la posibilidad de modelar las palabras y sus significados como estados dentro del propio espacio de Hilbert [42].

7.3.1. Codificación multidimensional: Forma y Contenido

En un espacio semántico tradicional, el significado de una palabra adquiere "sustancia" al ubicarse en unas coordenadas determinadas que definen su forma. Sin embargo, al restringirse al uso exclusivo de los números reales, el PLN clásico queda comúnmente limitado a capturar únicamente su *semántica distribucional* (las estadísticas y frecuencias con las que coocurre junto a otras palabras).

Al trasladar el procesamiento de lenguaje al dominio cuántico de Hilbert, las propiedades exclusivas de los números complejos permiten unificar distintos niveles de información semántica en un único vector de estado. Esto logra vincular el uso externo del lenguaje (su *forma*) directamente con su fundamento conceptual intrínseco (su *contenido*). Para ello, se propone una división explícita de las dimensiones de almacenamiento:

- **Componente Real:** Se utiliza de la misma forma para codificar la *semántica distribucional*. Captura el uso estadístico clásico, determinando el comportamiento de la palabra estrictamente en base a su contexto y al número de interacciones o apariciones que realiza con respecto a las palabras que le rodean en los distintos documentos analizados.
- **Componente Imaginaria:** Se reserva de forma ortogonal para alojar la *semántica referencial u ontológica*. Representa los fundamentos preexistentes, los conceptos formales o el núcleo estricto que designa la propia palabra (el significado que podría extraerse de ontologías taxonómicas u organizaciones como base de conocimientos, por ejemplo *WordNet* o diagramas *SNOMED-CT*).

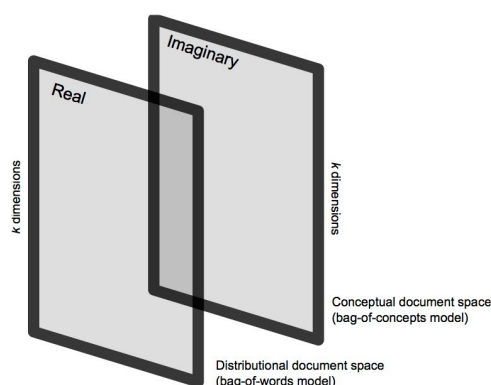


Figura 7.5: Esquema del espacio documental complejo vectorial unificando la semántica distribucional y referencial en el espacio de Hilbert. Fuente: [42].

7.3.2. Afinidad Cuántica, Superposiciones y Fases

En el apartado clásico se demostró que la afinidad semántica puede medirse mediante distintas distancias sobre un espacio real continuo. No obstante, al emplear el espacio de Hilbert esta medición no solo relaciona dos palabras distintas, sino que va un paso más allá. Al tratar con números complejos, es posible medir la distancia cuantitativa que existe entre la parte estadística de uso de la palabra (su componente real) y su definición conceptual referencial alojada en su componente imaginaria.

Para este análisis se evalúa la **fase o ángulo** que se forma interactivamente entre ambas componentes, logrando que el producto interno refleje tanto la componente real como imaginaria del contenido.

Capítulo 8

Implementación Clásica: Algoritmo Word2Vec

Word2Vec, cuyo nombre proviene de la contracción de *Word to Vector*, es un modelo de ML que permite representar palabras como vectores de grandes dimensiones. Su principal virtud reside en que las relaciones semánticas y sintácticas entre palabras quedan codificadas como proximidad geométrica entre dichos vectores, permitiendo así capturar el significado de forma numérica y operable.

Fue desarrollado y publicado por Google en 2013 [13], además en esa misma fecha, se introdujo este modelo en su motor de búsqueda. Desde entonces, el modelo ha sido ampliamente adoptado y ha dado lugar a múltiples mejoras y variantes de rendimiento.

En este capítulo se explorarán la arquitectura del modelo, su función de optimización y el proceso de entrenamiento que permite generar el espacio vectorial latente en el que las palabras quedan representadas.

8.1. Arquitectura del modelo

Word2Vec aprende las representaciones de las palabras procesando grandes volúmenes de texto y capturando el contexto en el que aparece cada término [20]. Su arquitectura se fundamenta en una red neuronal superficial (*shallow neural network*), que a diferencia de las redes profundas (*deep neural networks*), dispone únicamente de una capa oculta entre la capa de entrada y la de salida. Este diseño proporciona una ventaja clave: permite entrenar el modelo de forma eficiente sobre corpus de gran tamaño, manteniendo un coste computacional razonable.

El resultado del entrenamiento es la asignación de un vector de cientos de dimensiones a cada palabra del vocabulario. La posición de cada vector en ese espacio no es arbitraria, es decir, palabras empleadas en contextos similares acaban próximas entre sí, mientras que palabras semánticamente

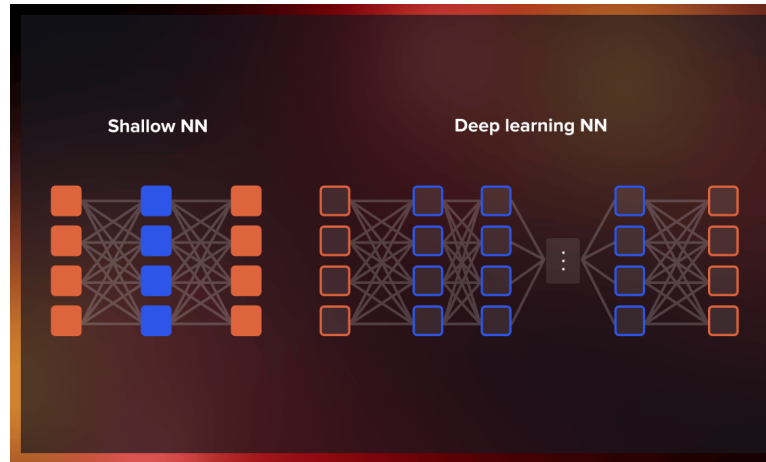


Figura 8.1: Comparativa entre una red neuronal superficial (Word2Vec) y una red neuronal profunda. Fuente: [20].

distantes se alejan geoméricamente.

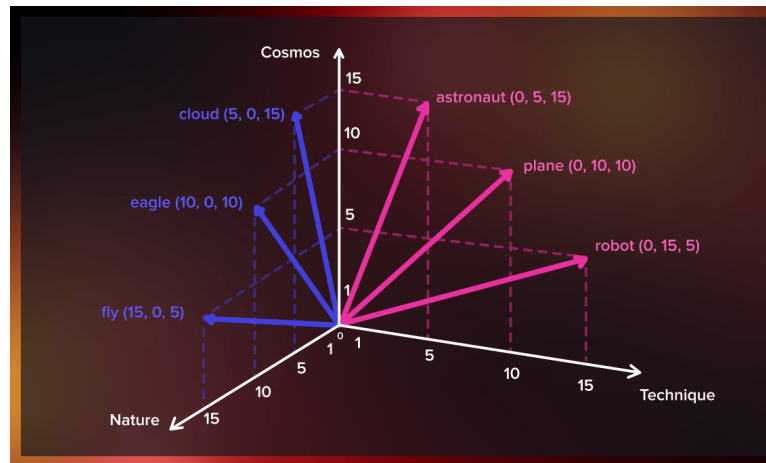


Figura 8.2: Representación vectorial de palabras en el espacio latente generado por Word2Vec. Fuente: [20].

Una consecuencia directa de esta estructura es que los vectores admiten operaciones algebraicas con significado semántico. El ejemplo más conocido de esta propiedad es la siguiente analogía entre género y realeza:

$$\vec{v}(\text{Rey}) - \vec{v}(\text{Hombre}) + \vec{v}(\text{Mujer}) \approx \vec{v}(\text{Reina})$$

En esta expresión, al restar al vector de *Rey* la componente que codifica la masculinidad y sumar la que codifica la feminidad, el vector resultante converge hacia la representación de *Reina*. Esta propiedad es inherente a

la forma natural del entrenamiento: como *Rey* y *Reina* comparten contextos de realeza pero difieren en los contextos de género, el modelo aprende implícitamente a separar ambas dimensiones dentro del espacio vectorial.

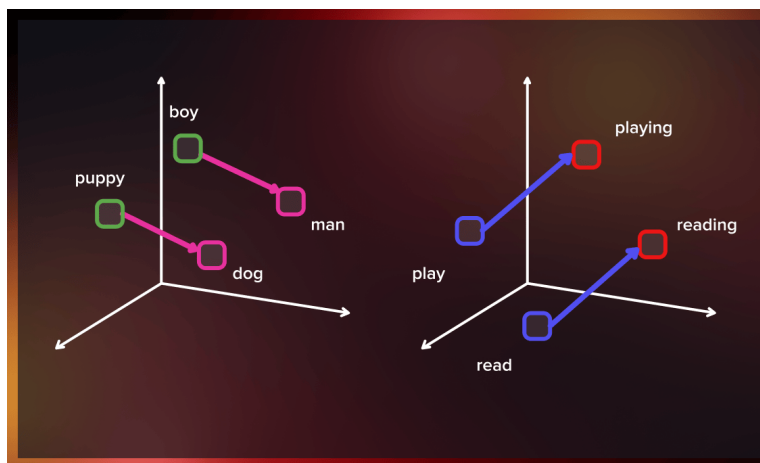


Figura 8.3: Representación geométrica de las similitudes semánticas entre palabras en el espacio vectorial de Word2Vec. Fuente: [20].

8.1.1. Modelos de entrenamiento: CBOW y Skip-gram

Word2Vec ofrece dos variantes arquitectónicas para aprender los vectores de palabras: el modelo **Continuous Bag of Words (CBOW)** y el modelo **Skip-gram**. Ambos comparten la misma red neuronal superficial como base, pero se diferencian en el aprendizaje, qué se usa como entrada y qué se predice como salida.

En este trabajo se ha optado por implementar el modelo **Skip-gram**, ya que, como se verá en los capítulos siguientes, sus características lo hacen especialmente adecuado para la adaptación al entorno cuántico propuesto.

Continuous Bag of Words (CBOW)

El modelo CBOW toma como entrada las palabras del contexto que rodean a una posición central y predice cuál es la palabra que debería ocupar dicha posición. En otras palabras, dadas las palabras vecinas, el modelo intenta adivinar la palabra central.

Este enfoque resulta eficiente cuando el corpus es grande y las palabras frecuentes tienen representaciones ricas. Sin embargo, tiende a promediar la información del contexto, lo que puede penalizar los valores semánticos de palabras menos frecuentes.

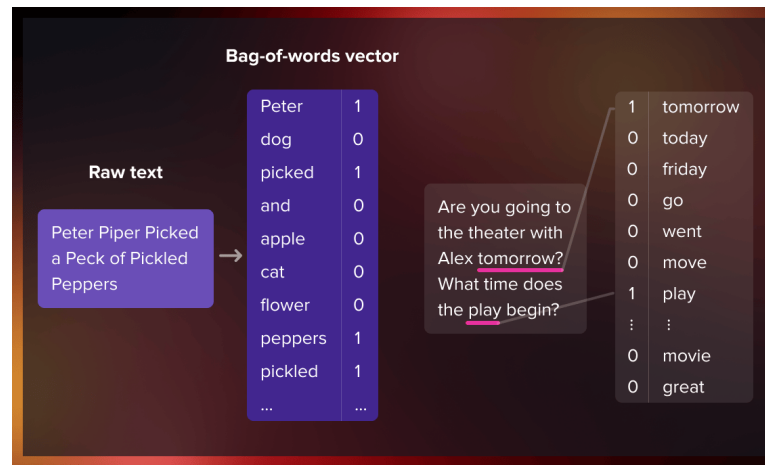


Figura 8.4: Ejemplo del funcionamiento del modelo CBOW: a partir de las palabras de contexto circundantes se predice la palabra objetivo central. Fuente: [20].

Skip-gram

Skip-gram es un modelo diseñado para hacer lo opuesto a CBOW. En lugar de predecir la palabra central a partir del contexto que tiene en su ventana, toma una única palabra central como entrada e intenta predecir las palabras que aparecen a su alrededor.

El proceso de entrenamiento de esta red neuronal superficial funciona de la siguiente manera: dada la palabra central (la palabra de entrada), se selecciona aleatoriamente una palabra de su contexto inmediato. La red calcula cuál es la probabilidad de que cada palabra del vocabulario sea una palabra cercana.^a la palabra de entrada [21].

Los resultados de estas probabilidades indican cuán probable es que cada término del vocabulario aparezca en las proximidades de la palabra de entrada. Por ejemplo, si la palabra de entrada proporcionada a la red ya entrenada es *reino*, las probabilidades serán considerablemente mayores para palabras como *España* o *Inglaterra* que para términos sin relación semántica como *lápiz* o *mango*.

Detalles del modelo básico Skip-gram

Dado que una red neuronal no puede procesar cadenas de texto directamente, cada palabra se representa mediante un *vector one-hot* (una forma de codificación sencilla aunque no la única). Consiste en un vector de tantas componentes como palabras tiene el vocabulario (por ejemplo, 10.000), donde únicamente la posición correspondiente a la palabra de entrada vale 1 y el resto valen 0.

La capa de salida de la red produce también un vector de 10.000 componentes, donde cada valor refleja la probabilidad de que la palabra correspondiente sea una palabra cercana a la entrada. No existe función de activación en la capa oculta, pero la capa de salida aplica *softmax* para normalizar las probabilidades.

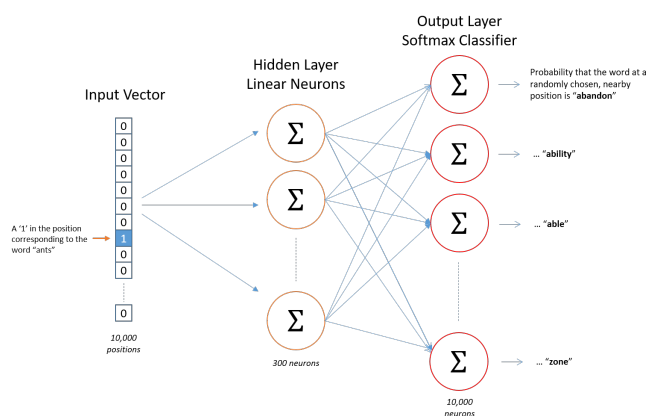


Figura 8.5: Arquitectura de la red neuronal del modelo Skip-gram. Fuente: [21].

Durante el entrenamiento, tanto la entrada como la salida de la red son vectores *one-hot*. Sin embargo, al evaluar la red con una palabra de entrada, el vector de salida ya no es *one-hot*, sino una distribución de probabilidad sobre todo el vocabulario.

A modo de ejemplo concreto, supóngase que la **capa oculta** aprende vectores de palabras de 300 dimensiones. En ese caso, los pesos de dicha capa quedan representados por una matriz de 10.000×300 : una fila por cada palabra del vocabulario y una columna por cada neurona oculta. Este es precisamente el tamaño que empleó Google en el trabajo original en el que presentó Word2Vec [13].

Al final, el objetivo de este modelo es aprender la matriz de pesos de la capa oculta. Una vez se tiene esta matriz, la capa de salida es inmediata.

Llegados a este punto, uno puede preguntarse cuál es la utilidad de representar las palabras mediante vectores de todo menos una única posición con valor 1. La respuesta reside en la eficiencia, multiplicar un vector *one-hot* de 1×10.000 por la matriz de 10.000×300 selecciona de forma inmediata la fila correspondiente a la palabra de entrada, sin necesidad de realizar el producto matricial completo.

Con esto se concluye que la capa oculta del modelo actúa en realidad como una tabla *lookup*: la salida de dicha capa es directamente el vector de representación de la palabra de entrada. Así, el vector de 1×300 resultante se pasa a la **capa de salida**, que es un clasificador de regresión *softmax*.

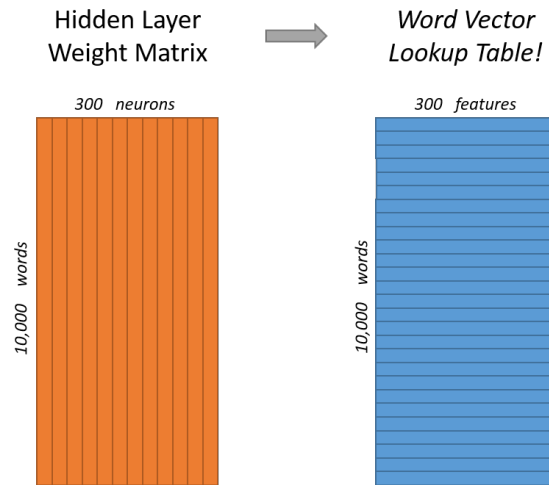


Figura 8.6: Matriz de pesos de la capa oculta del modelo Skip-gram: 10.000 filas (una por palabra del vocabulario) y 300 columnas (una por neurona oculta). Cada fila constituye el vector de representación de su palabra correspondiente. Fuente: [21].

$$[0 \ 0 \ 0 \ 1 \ 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = [10 \ 12 \ 19]$$

Figura 8.7: Multiplicación de un vector *one-hot* por la matriz de pesos de la capa oculta, obteniendo directamente el vector de representación de la palabra de entrada. Fuente: [21].

Cada neurona de salida produce un valor entre 0 y 1 aplicando la función $\exp(x)$ sobre el producto de su vector de pesos y el vector proveniente de la capa oculta. Para que todos los valores sumen 1 y formen una distribución de probabilidad, cada resultado se divide entre la suma de los valores de las 10.000 neuronas de salida [21].

Mejoras aplicadas al modelo básico Skip-gram

Retomando el ejemplo anterior, con 300 dimensiones y un vocabulario de 10.000 palabras, la red neuronal dispone de dos matrices de pesos: una correspondiente a la capa oculta y otra a la capa de salida. Cada una de ellas contiene un total de $300 \times 10.000 = 3$ millones de parámetros. Para ajustar correctamente este volumen de parámetros y evitar al mismo tiempo el sobreajuste se debe disponer de conjuntos de datos de entrenamiento de gran tamaño, lo que convierte el entrenamiento del modelo básico en un

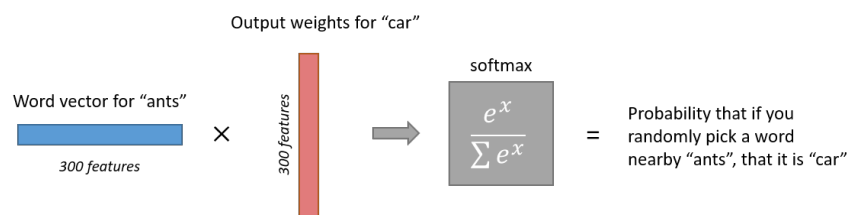


Figura 8.8: Cálculo de la salida de la neurona correspondiente a la palabra *car* en la capa *softmax* del modelo Skip-gram. Fuente: [21].

proceso computacionalmente costoso.

Para abordar este problema, Google publicó un segundo artículo [23] en el que se proponían dos innovaciones clave sobre el modelo básico:

- **Submuestreo de palabras frecuentes:** se reduce la frecuencia con la que las palabras muy comunes participan en el entrenamiento, disminuyendo así el número total de ejemplos procesados.
- **Negative Sampling:** se sustituye la función de optimización original (descenso del gradiente) por una alternativa más eficiente que, en lugar de actualizar todos los pesos de la red en cada iteración, únicamente modifica un subconjunto reducido de ellos.

Estas dos mejoras demostraron que, lejos de comprometer la calidad del modelo, se conseguía una reducción del coste computacional además de una mejora de la calidad de los vectores de palabras resultantes. Dado que en la implementación práctica de este trabajo se ha empleado el *Negative Sampling* como función de optimización, su funcionamiento se describe en detalle en la sección siguiente.

8.2. Función de optimización

Para comprender la mejora que introduce el *Negative Sampling*, es necesario partir de la formulación matemática original del modelo básico de Skip-gram y, a partir de ella, entender qué limitaciones motivan la adopción de esta nueva función de optimización [38].

El objetivo principal de Skip-gram es maximizar la probabilidad logarítmica promedio sobre todo el corpus de entrenamiento, lo que queda expresado mediante la siguiente fórmula:

$$\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} | w_t)$$

Donde:

- c es el tamaño de la ventana de contexto. Valores más altos de c generan más ejemplos de entrenamiento y, por tanto, ofrecen mayor precisión, aunque a costa de un tiempo de entrenamiento superior.
- T es el número total de palabras del corpus de entrenamiento.
- w_t es la palabra de entrenamiento en la posición t .

La probabilidad condicional $p(w_{t+j} \mid w_t)$ del modelo básico de Skip-gram se define mediante la función *softmax* de la siguiente forma [38]:

$$p(w_O \mid w_I) = \frac{\exp(v'_{w_O}{}^\top v_{w_I})}{\sum_{w=1}^W \exp(v'_w{}^\top v_{w_I})}$$

Donde v_w y v'_w son, respectivamente, los vectores de representación de *entrada* y de *salida* de la palabra w , y W es el número total de palabras del vocabulario.

Esta formulación presenta un problema de escalabilidad: el coste de calcular $\nabla \log p(w_O \mid w_I)$ es proporcional a W , cuyo valor suele ser muy elevado (entre 10^5 y 10^7 términos). Ello implica que, en el denominador del *softmax*, es necesario acumular los valores exponenciales de todas las palabras del vocabulario en cada paso de entrenamiento, lo que hace que el proceso resulte prohibitivamente lento cuando el vocabulario es de gran escala. Es precisamente este coste el que motiva la introducción del *Negative Sampling* como función de optimización alternativa.

Esta nueva función de optimización persigue maximizar la similitud entre las palabras que aparecen en el mismo contexto y minimizarla cuando proceden de contextos distintos. Sin embargo, en lugar de aplicar esa minimización sobre todas las palabras del corpus, selecciona aleatoriamente k palabras, denominadas *muestras negativas*, y las emplea para optimizar la función objetivo [18]. La expresión de dicha función es la siguiente:

$$\log \sigma(v'_{w_O}{}^\top v_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-v'_{w_i}{}^\top v_{w_I}) \right]$$

Donde σ es la función sigmoide y $P_n(w)$ es la distribución de ruido a partir de la cual se extraen las muestras negativas. Esta distribución se calcula como la distribución unigrama de las palabras elevada a la potencia $\frac{3}{4}$, exponente elegido empíricamente en el trabajo original donde se introduce este algoritmo [23]:

$$P_n(w) = \frac{U(w)^{\frac{3}{4}}}{Z}$$

Donde $U(w)$ es la frecuencia de aparición de la palabra w en el corpus y Z es la constante de normalización que garantiza que la distribución sume 1.

Maximizar esta expresión conlleva maximizar el producto escalar en su primer término y minimizarlo en el segundo. En otras palabras, las palabras que aparecen en el mismo contexto se ven forzadas a adquirir representaciones vectoriales más similares entre sí, mientras que las que pertenecen a contextos distintos se ven forzadas a tener vectores menos similares [18].

En cuanto a la elección del hiperparámetro k , el segundo artículo de Google sobre Word2Vec [23] indica que un valor en el rango de 5 a 20 es adecuado para corpus de entrenamiento de tamaño reducido, mientras que para corpus de gran escala se recomienda utilizar valores de entre 2 y 5.

Capítulo 9

Implementación Cuántica: Algoritmo QWord2Vec

En el capítulo anterior se describió en detalle el algoritmo Word2Vec clásico y su variante Skip-gram, que emplea redes neuronales superficiales para aprender representaciones vectoriales de palabras a partir de grandes corpus de texto. En este capítulo se presenta *QWord2Vec*, una adaptación cuántica de dicho modelo propuesta por Ohno [24], cuyo objetivo es explorar si el uso de circuitos cuánticos paramétricos puede ofrecer ventajas en la calidad de las representaciones vectoriales de palabras.

En este capítulo se explorarán la arquitectura del circuito cuántico, la función de coste empleada para su optimización y la estrategia híbrida clásico-cuántica utilizada durante el entrenamiento.

9.1. Adaptación del algoritmo al entorno cuántico

El circuito cuántico de *QWord2Vec* sigue la misma estructura de Word2Vec, compuesta por una parte de **codificación** y una parte de **decodificación**. Está formado por puertas de rotación de Pauli (unitarias parametrizadas) y puertas de Pauli-X controladas para introducir entrelazamiento, organizadas en una estructura de capas con profundidad de circuito ajustable. Dado un vector *one-hot* como entrada para una palabra y un vector de salida que representa su contexto [24].

9.2. Diseño del Circuito Cuántico Parametrizado

Como se indicó con anterioridad, la arquitectura de *QWord2Vec* mantiene una estructura análoga a la del algoritmo clásico, dividiéndose en una fase de codificación y otra de decodificación. Esta similitud estructural se ilustra en la Figura 9.1.

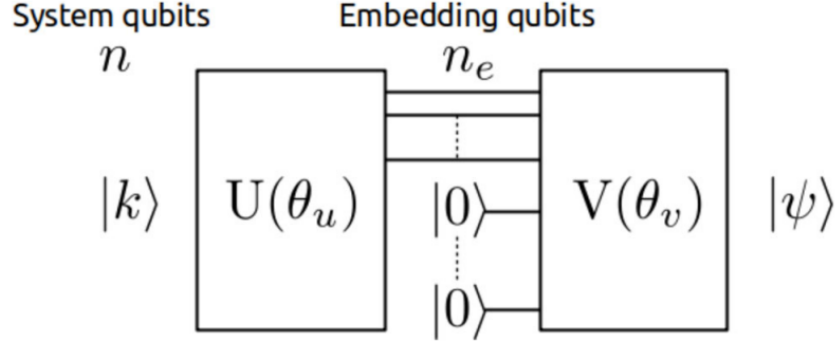


Figura 9.1: Estructura de QWord2Vec. Fuente: [24].

En la imagen anterior, se aprecia como hay dos matrices parametrizadas, $U(\theta_u)$ y $V(\theta_v)$ junto a una matriz unitaria de entrelazamiento U_{enta} . En este sistema, n denota el número total de qubits, mientras que n_e indica el número de qubits destinados específicamente a la representación de los *embeddings*, debiendo cumplirse la restricción $1 \leq n_e \leq n$.

La entrada al circuito viene dada por el estado cuántico $|k\rangle$, el cual representa a una palabra codificada mediante un vector *one-hot* Word2Vec donde $0 \leq k \leq 2^n - 1$. A partir de este estado inicial, el sistema evoluciona hacia un estado de salida $|\psi\rangle$. Parecido al mapeo directo que realiza Word2Vec, el mapeo que hay entre $|k\rangle$ y $|\psi\rangle$, lo realizan los bloques U y V .

Los estados cuánticos de salida resultantes codifican la información correspondiente al contexto de la palabra de entrada, cuya extracción se lleva a cabo realizando mediciones en la base computacional (observable Pauli Z).

El modelo $U(\theta_u)$ y $V(\theta_v)$ utilizan las matrices unitarias parametrizadas y las matrices unitarias (U_{enta}) para proporcionar el entrelazamiento cuántico de la siguiente manera:

$$U(\theta) = \prod_{l=1}^L U_{\text{enta}} R_Z(\theta_z^{L-l+1})^{\otimes n} R_Y(\theta_y^{L-l+1})^{\otimes n} R_X(\theta_x^{L-l+1})^{\otimes n}$$

donde L representa la longitud del circuito cuántico. Por simplicidad, se asumirá que tanto $U(\theta_u)$ como $V(\theta_v)$ poseen la misma profundidad, a menos que se indique explícitamente lo contrario.

Para implementar U_{enta} , se ha usado puertas controladas Pauli X (CNOT), siguiendo una configuración de circuito por bloques [32]. Esta disposición favorece notablemente la expresividad del circuito y maximiza su capacidad para generar entrelazamiento. La Figura 9.2 muestra visualmente la estructura del circuito resultante al combinar las rotaciones de Pauli con el bloque

de entrelazamiento descrito.

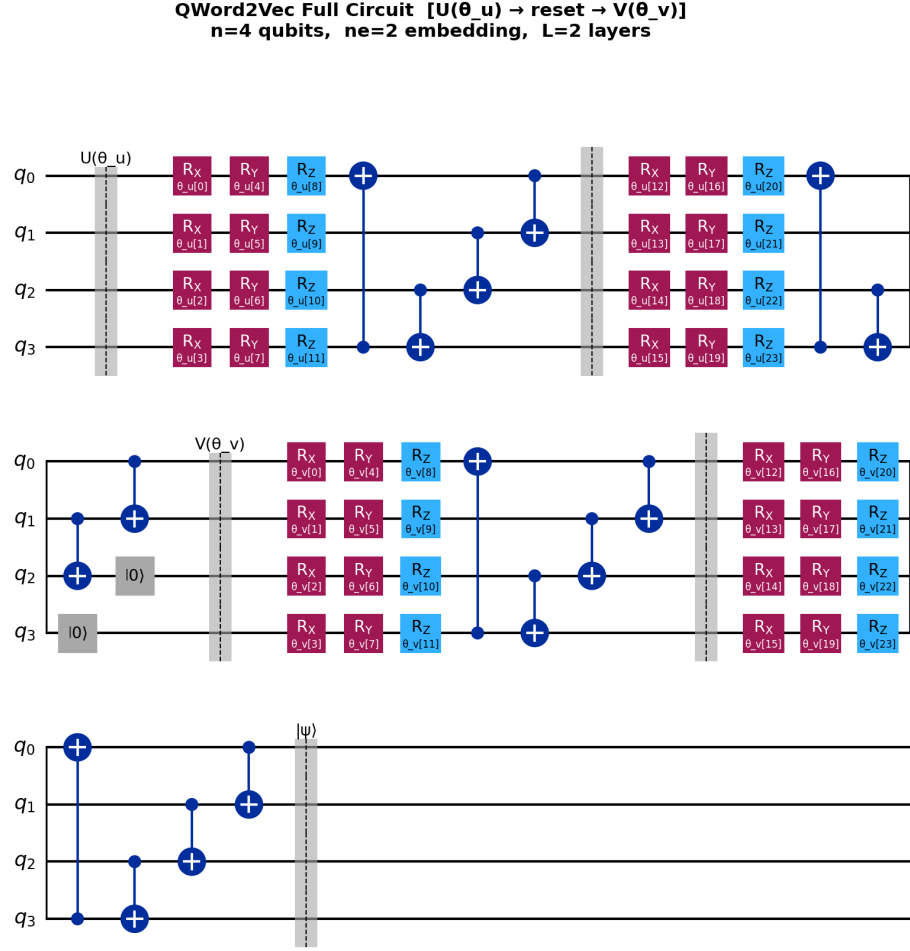


Figura 9.2: Circuito cuántico parametrizado de QWord2Vec con puertas de rotación de Pauli (R_X, R_Y, R_Z) y entrelazamiento con configuración por bloques mediante puertas CNOT.

El uso combinado de las tres puertas de rotación (R_X, R_Y, R_Z) permite que los estados cuánticos que representan a las distintas palabras alcancen una mayor dispersión en el espacio de Hilbert, superando las limitaciones de representatividad que se obtendrían empleando una sola matriz de rotación, tal y como se observa en la Figura 9.3.

9.2.1. Estimación de la longitud del circuito

Con el fin de acotar el espacio de búsqueda durante el ajuste de hiperparámetros, se emplea la fórmula heurística propuesta en [24] para estimar la profundidad L del circuito cuántico:

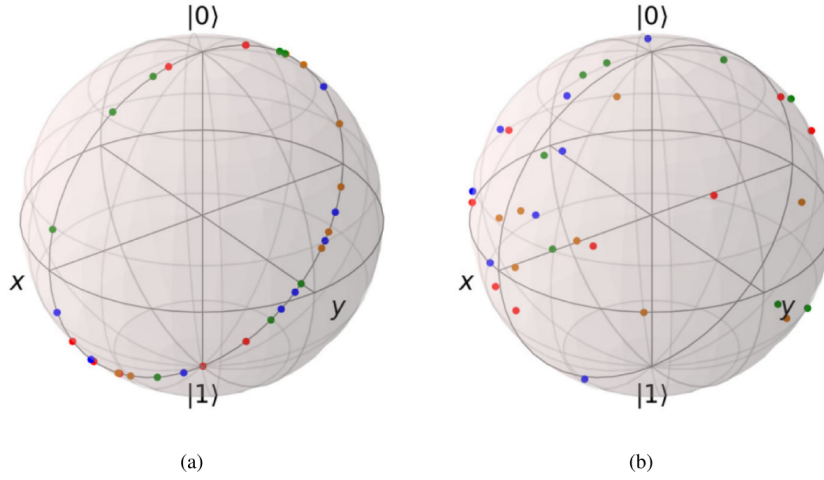


Figura 9.3: Representación de las palabras sobre la esfera de Bloch según las puertas de rotación empleadas en $U(\theta_u)$: (a) usando únicamente R_Y , y (b) usando R_X , R_Y y R_Z combinadas. Los puntos coloreados corresponden a las palabras y el número total de puntos es 32. El uso de las tres puertas de rotación de Pauli permite una mayor dispersión de las representaciones a lo largo del espacio de Hilbert. Fuente: [24].

$$L = \left\lceil \frac{\#data}{3(n + n_e)A_{th}} p(n, n_e) \right\rceil$$

donde los términos que intervienen en la expresión son:

- $\#data$: número de pares de entrenamiento disponibles.
- n : número total de qubits del sistema.
- n_e : número de qubits de embeddings.
- A_{th} : umbral de tolerancia determinado de forma empírica.
- $p(n, n_e) \in [0, 0,5]$: probabilidad de error en función de n y n_e .

La expresión determina la profundidad mínima necesaria para que la cantidad promedio de datos por parámetro no supere el umbral A_{th} . Dado que cada capa aporta $3(n + n_e)$ parámetros entrenables (un parámetro entrenable por cada matriz de rotación de Pauli), la fórmula divide la carga total de entrenamiento entre dicha capacidad ponderada por el umbral. La función techo $\lceil \cdot \rceil$ garantiza que L tome un valor entero.

La probabilidad de error $p(n, n_e)$ se obtiene a partir de la siguiente inecuación la cual relaciona la probabilidad de error entre n y n_e :

$$n_e \geq (1 - H(p)) \cdot 2^n$$

donde $H(p)$ es la función de entropía binaria, representada en la Figura 9.4.

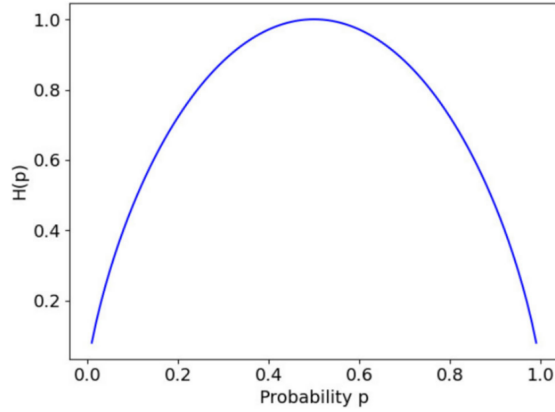


Figura 9.4: Entropía binaria $H(p)$, donde p denota la probabilidad de error empleada en este estudio. Fuente: [24].

9.3. Definición de la función de evaluación y pérdida usadas

A continuación se procederá a explicar cuales han sido las funciones de evaluación y de pérdidas que han sido utilizadas para entrenar a este modelo ya que difieren completamente de la aproximación clásica seguida.

9.3.1. Evaluación de los datos de embeddings

Para evaluar la calidad de los *embeddings* generados por QWord2Vec, se emplea el coeficiente de correlación de Pearson, calculado a partir de las distancias entre los vectores producidos por ambos modelos, Word2Vec y QWord2Vec.

Una vez completado el entrenamiento, se realiza una medición sobre los qubits destinados a los *embeddings*, obteniendo así los vectores clásicos correspondientes a cada palabra. A partir de estos vectores, se construye un *vector distancia*, cuyas componentes son las distancias euclídeas entre los vectores de *embeddings* de todas las palabras del vocabulario. Este proceso se aplica tanto al modelo clásico como al cuántico, tomando el vector distancia de Word2Vec como referencia de comparación (*ground truth*).

El motivo por el que no se utiliza únicamente la entropía cruzada como métrica de evaluación reside en que los circuitos cuánticos con un elevado número de parámetros tienden a reducir su valor. Por ello, se añade el coeficiente de correlación de Pearson sobre los vectores distancia ya que constituye una métrica más adecuada y robusta para valorar la capacidad del modelo. En la siguiente subsección se describe en detalle la función de pérdida empleada durante el entrenamiento del circuito.

9.3.2. Función de pérdida

La función de pérdida usada, además de tener el componente de la pérdida de entropía cruzada, se le añade un término junto al coeficiente de correlación de Pearson. Por lo que la función de pérdida queda de la siguiente manera:

$$\text{Loss} := \text{PérdidaEntropíaCruzada} - C \cdot \text{CoeficienteCorrelación}(\text{DatosDeEmbeddings}, \text{EtiquetasEntrenamiento})$$

Donde $C > 0$ es un parámetro de balanceo. El operador que acompaña a dicho parámetro, se refiere al coeficiente de correlación entre el vector distancia de los datos de embeddings (*DatosDeEmbeddings*) en QWord2Vec. Este se actualiza en cada iteración mientras que *EtiquetasEntrenamiento* se ha obtenido de la siguiente forma:

dado cada palabra objetivo del corpus, se han establecido a 0.5 las dos palabras inmediatamente adyacente a la palabra de referencia, mientras que el resto de posiciones del vector de etiqueta de tamaño $N = 2^n$ se establecen a cero. De este modo, el vector resultante suma 1 y puede interpretarse como una distribución de probabilidad uniforme sobre los dos contextos posibles de cada palabra objetivo

Para un vector de tamaño N correspondiente a $|k\rangle$

9.4. Estrategia de optimización

El algoritmo de optimización empleado en QWord2Vec es QN-SPSA, propuesto por Gacon et al. [11]. En cada paso de optimización, requiere únicamente 2 ejecuciones del circuito cuántico para estimar el gradiente y otras 4 para aproximar el tensor métrico de Fubini-Study, con un coste independiente del número de parámetros del circuito. Esta capacidad de escalado lo convierte en un candidato especialmente adecuado para tareas de optimización en dispositivos NISQ.

Para entender el funcionamiento de QN-SPSA, es necesario introducir previamente los algoritmos en los que se basa [7].

9.4.1. Funcionamiento del algoritmo

Los optimizadores empleados en CVCs operan bajo el mismo principio que los utilizados en ML clásico: en cada iteración estiman el gradiente de la función de coste y, a partir de este, actualizan los parámetros del modelo con el objetivo de minimizar dicha función.

El punto de partida es el Gradient Descent (GD), cuya regla de actualización de parámetros es:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \nabla f(\mathbf{x}^{(t)})$$

donde f es la función de pérdida, $\mathbf{x}$ el vector de parámetros, η la tasa de aprendizaje y el superíndice t la iteración actual. El gradiente ∇f se calcula dimensión a dimensión, lo que implica $\mathcal{O}(d)$ mediciones cuánticas, siendo d el número de parámetros entrenables. Dado el elevado coste de las mediciones en un circuito cuántico, el GD resulta inviable para circuitos con un número alto de parámetros así como un gran número de iteraciones para la optimización.

Para resolver este problema de escalado, surgió el optimizador Simultaneous Perturbation Stochastic Approximation (SPSA) [41]. En lugar de calcular el gradiente componente a componente, SPSA selecciona una dirección aleatoria $\mathbf{h} \in U(\{-1, 1\}^d)$, donde $U(\{-1, 1\}^d)$ es una distribución uniforme discreta de d dimensiones, y estima el gradiente proyectado mediante una aproximación de diferencias finitas con perturbación ϵ :

$$\nabla_{\mathbf{h}} f(\mathbf{x}) \equiv \mathbf{h} \cdot \nabla f(\mathbf{x}) \simeq \frac{1}{2\epsilon} (f(\mathbf{x} + \epsilon \mathbf{h}) - f(\mathbf{x} - \epsilon \mathbf{h})).$$

A partir de esta expresión se construye el estimador estocástico del gradiente:

$$\widehat{\nabla} f(\mathbf{x}, \mathbf{h})_{\text{SPSA}} = \nabla_{\mathbf{h}} f(\mathbf{x}) \mathbf{h},$$

con el que la regla de actualización de parámetros queda:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \widehat{\nabla} f(\mathbf{x}^{(t)}, \mathbf{h}^{(t)})_{\text{SPSA}},$$

donde $\mathbf{h}^{(t)}$ se muestrea en cada iteración. Al tratarse de una aproximación estocástica, el gradiente estimado no es exacto, pero se ha demostrado que el algoritmo converge de forma efectiva cuando se realizan suficientes iteraciones de optimización.

Sin embargo, SPSA hereda del GD el problema de operar en el espacio euclídeo de los parámetros, lo que hace que el resultado de la optimización dependa de la parametrización elegida. Para eliminar esta dependencia, Amari (1998) propuso el algoritmo clásico *Natural Gradient Descent*, que reformula el problema de optimización en términos de distribuciones de probabilidad sobre las posibles salidas del modelo. De este modo, cada paso

de optimización resulta invariante frente a reparametrizaciones, ya que se realiza en el espacio de distribuciones en lugar del espacio de parámetros. Su regla de actualización incorpora la matriz de información de Fisher F como tensor métrico:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta F^{-1} \nabla f(\mathbf{x}^{(t)}).$$

La matriz de información de Fisher actúa como un tensor métrico que transforma los pasos de descenso realizados en el espacio euclídeo de los parámetros en pasos de descenso en el espacio de distribuciones de probabilidad.

La versión cuántica de este algoritmo, Quantum Natural Gradient Descent (QNG) [35], traslada este mismo principio al espacio de estados cuánticos. En dicho espacio, el tensor métrico invariante es el **tensor de Fubini-Study** $\mathbf{g}$, de dimensiones $d \times d$, definido como:

$$g_{ij}(\mathbf{x}) = -\frac{1}{2} \frac{\partial}{\partial \mathbf{x}_i} \frac{\partial}{\partial \mathbf{x}_j} F(\mathbf{x}', \mathbf{x}) \Big|_{\mathbf{x}'=\mathbf{x}},$$

donde $F(\mathbf{x}', \mathbf{x}) = |\langle \phi(\mathbf{x}') | \phi(\mathbf{x}) \rangle|^2$ y $\phi(\mathbf{x})$ es el circuito ansatz parametrizado con entrada $\mathbf{x}$. La regla de actualización de QNG es:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \mathbf{g}^{-1}(\mathbf{x}^{(t)}) \nabla f(\mathbf{x}^{(t)}).$$

El uso del tensor $\mathbf{g}$ permite una convergencia más rápida y hacia mejores mínimos que el GD. No obstante, su cálculo requiere un gran número de mediciones cuánticas, lo que lo hace igualmente inescalable para circuitos de gran tamaño y con gran número de iteraciones para optimizarlos.

QN-SPSA surge precisamente como solución a esta limitación, combinando las ventajas de SPSA y QNG. Estima de forma estocástica tanto el gradiente como el tensor de Fubini-Study, manteniendo un coste constante independiente de d . El gradiente se estima con el mismo procedimiento que en SPSA, mientras que el tensor se aproxima mediante un estimador de segundo orden con dos perturbaciones estocásticas adicionales:

$$\widehat{\mathbf{g}}(\mathbf{x}, \mathbf{h}_1, \mathbf{h}_2)_{\text{SPSA}} = \frac{\delta F}{8\epsilon^2} \left(\mathbf{h}_1 \mathbf{h}_2^\top + \mathbf{h}_2 \mathbf{h}_1^\top \right),$$

donde

$$\delta F = F(\mathbf{x}, \mathbf{x} + \epsilon \mathbf{h}_1 + \epsilon \mathbf{h}_2) - F(\mathbf{x}, \mathbf{x} + \epsilon \mathbf{h}_1) - F(\mathbf{x}, \mathbf{x} - \epsilon \mathbf{h}_1 + \epsilon \mathbf{h}_2) + F(\mathbf{x}, \mathbf{x} - \epsilon \mathbf{h}_1),$$

y $\mathbf{h}_1, \mathbf{h}_2 \in U(\{-1, 1\}^d)$ son dos direcciones de perturbación muestreadas aleatoriamente. La regla de actualización de parámetros resultante es:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \widehat{\mathbf{g}}^{-1}(\mathbf{x}^{(t)}, \mathbf{h}_1^{(t)}, \mathbf{h}_2^{(t)})_{\text{SPSA}} \widehat{\nabla} f(\mathbf{x}^{(t)}, \mathbf{h}^{(t)})_{\text{SPSA}}.$$

Cada paso de optimización t requiere tres direcciones de perturbación $\mathbf{h}^{(t)}, \mathbf{h}_1^{(t)}, \mathbf{h}_2^{(t)}$, lo que se traduce en 2 evaluaciones del circuito para el gradiente y 4 para el tensor, con una complejidad total de $\mathcal{O}(1)$ respecto al número de parámetros. Esta propiedad lo convierte en el optimizador idóneo para su uso en dispositivos NISQ. La Figura 9.5 muestra una comparativa del comportamiento de los distintos algoritmos de optimización descritos.

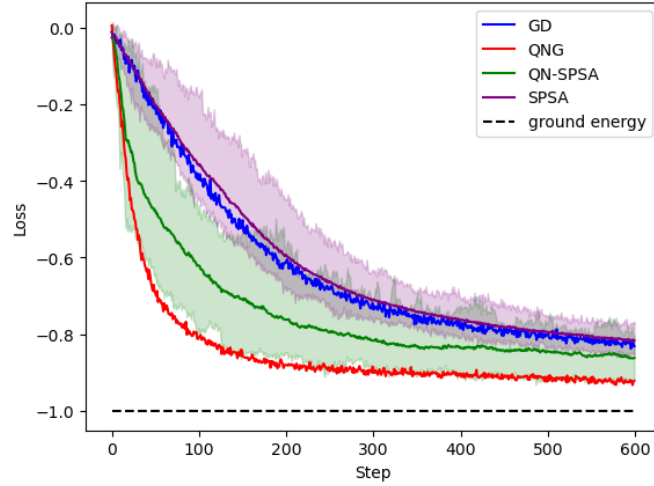


Figura 9.5: Comparativa de los algoritmos de optimización GD, SPSA, QNG y QN-SPSA en términos de convergencia y coste computacional. Fuente: [7].

Como se puede apreciar, en este caso sale que el que mejor resultados tiene es QNG. Sin embargo aquí no se está contemplando el tiempo de ejecución, que como se comentó, este optimizador es menos eficiente y malo escalando (aquí se tienen pocas iteraciones). Con esto se demuestra como el QN-SPSA es el que mejor un mejor resultado ofrece de manera equilibrada, teniendo un loss bajo así como un tiempo de ejecución también menor y siendo una opción buena en cuanto a la escalabilidad.

Capítulo 10

Experimentación y Análisis de Resultados

Una vez descritos los fundamentos teóricos de Word2Vec y QWord2Vec, este capítulo recoge el proceso de experimentación llevado a cabo para comparar ambos modelos. Se han desarrollado las implementaciones de ambos algoritmos en el lenguaje de programación *Python*.

Para el modelo clásico, se ha empleado la implementación disponible en la librería *Gensim*, que incluye tanto la arquitectura del modelo como la función de optimización Negative Sampling, desarrollada siguiendo las especificaciones originales de Google. Por su parte, la versión cuántica ha sido implementada utilizando la librería *Qiskit*, que proporciona las herramientas necesarias para la construcción del circuito parametrizado, la aplicación del optimizador descrito en el capítulo anterior, así como utilidades para la medición y transpilación del circuito.

El objetivo principal de esta experimentación es evaluar si los *embeddings* generados por QWord2Vec son capaces de preservar relaciones semánticas entre palabras de forma comparable a los producidos por Word2Vec. Para ello, se ha seguido una metodología sistemática que abarca desde la preparación del corpus y la configuración de los hiperparámetros, hasta el análisis de la función de pérdida durante el entrenamiento y la evaluación de la calidad de los *embeddings* mediante el coeficiente de correlación de Pearson. Asimismo, se incluye un análisis comparativo de los tiempos de ejecución de ambos modelos, dado que el coste computacional es uno de los factores críticos en el contexto de la computación cuántica simulada.

10.1. Metodología experimental

Esta sección describe los pasos previos necesarios para llevar a cabo la experimentación. En primer lugar, se presenta el corpus empleado para el entrenamiento de ambos modelos, junto con el proceso de preprocesamiento

aplicado en cada caso. A continuación, se detallan las herramientas y el entorno de ejecución utilizados, aspecto especialmente relevante en el caso de QWord2Vec, dado que la simulación clásica de circuitos cuánticos impone restricciones significativas sobre el tamaño del experimento realizable.

10.1.1. Dataset y preprocesamiento

Como se comentó anteriormente, la simulación clásica de circuitos cuánticos impone restricciones severas sobre el tamaño del experimento. A diferencia de Word2Vec clásico, que habitualmente se entrena con corpus de millones de frases y vocabularios de cientos de miles de palabras, el coste computacional de simular un circuito cuántico crece exponencialmente con el número de qubits, lo que hace inviable trabajar con corpus de gran escala en un entorno de computación personal que es con el que se ha trabajado.

Por este motivo, se ha adoptado el mismo corpus empleado en el trabajo de referencia [24], el cual fue tomado originalmente de una fuente web la cual ofrecía corpus pequeños para realizar evaluaciones de modelos de este estilo, solo que actualmente no está disponible la página web. El corpus consta de 13 frases en inglés con un vocabulario de 13 palabras únicas. Las frases utilizadas son las siguientes:

```
i like dog    i like cat    i like animal    dog cat animal
apple cat dog like    dog fish milk like    dog cat eyes
like
i like apple    apple i hate    apple i movie
book music like    cat dog hate    cat dog like
```

Las frases tienen una longitud de entre 3 y 4 palabras, lo que condiciona el tamaño de la ventana de contexto haciendo que sea pequeña. Desde el punto de vista de la distribución léxica, el corpus presenta un desequilibrio: las cuatro palabras más frecuentes (*like*, *dog*, *cat* e *i*) concentran el 67% de los tokens totales, mientras que seis palabras del vocabulario aparecen una única vez. Este desequilibrio puede introducir distorsiones en las relaciones semánticas aprendidas por el modelo, y es una limitación que se asume de forma consciente.

No obstante, el uso de este corpus está justificado por tres razones. En primer lugar, permite reproducir directamente las condiciones experimentales del paper de referencia, facilitando la comparación de resultados durante el desarrollo e implementación de variaciones que se han realizado. En segundo lugar, el objetivo del estudio no es obtener un modelo de procesamiento del lenguaje natural de calidad, sino evaluar si QWord2Vec es capaz de preservar las relaciones semánticas que aprende Word2Vec. Para este fin, un corpus pequeño y controlado es suficiente. En tercer lugar, el tamaño reducido del vocabulario permite inspeccionar e interpretar los *embeddings* obteni-

dos de forma directa, verificando manualmente que las relaciones semánticas resultantes son coherentes con el contenido del corpus.

En cuanto al preprocesamiento, no se ha realizado nada ya que el corpus viene preprocesado con sus palabras separadas por un espacio tal y como se espera sin necesidad de haber tenido que eliminar algún signo de puntuación o bien convertir todo a minúscula.

10.1.2. Entorno de ejecución y herramientas

Todos los experimentos se han llevado a cabo sobre un ordenador personal con sistema operativo Ubuntu, cuya compatibilidad con las librerías empleadas y su integración con el hardware disponible lo hacen especialmente adecuado para este tipo de desarrollo. Las características del hardware relevantes para el rendimiento son las siguientes:

- **Procesador:** Intel Core i7-13700F (30 MB de caché, frecuencia máxima superior a 5,2 GHz).
- **Memoria RAM:** 32 GB DDR4 a 3600 MHz.
- **Tarjeta gráfica:** MSI GeForce RTX 4070 con 12 GB de memoria GDDR6X.

En cuanto al software, las librerías principales utilizadas en el desarrollo son:

- **Python 3.10.12:** lenguaje de programación base del proyecto.
- **Gensim 4.4.0:** empleada para la implementación del modelo Word2Vec clásico.
- **Qiskit 1.4.5:** utilizada para la construcción y ejecución del circuito cuántico parametrizado.
- **Qiskit Aer 0.17.2:** backend de simulación cuántica sobre hardware clásico.

Respecto a la elección del simulador cuántico, *Qiskit* ofrece tres opciones: *AerSimulator*, *StatevectorSimulator* y *QasmSimulator*. Los dos últimos se encuentran en proceso de deprecación, dado que *AerSimulator* los unifica y supera en capacidades siendo el que más soporte y desarrollo recibe. Por este motivo, se ha optado por *AerSimulator* como backend de simulación para la ejecución de QWord2Vec.

10.2. Configuración de hiperparámetros

Tanto el modelo clásico como el cuántico presentan una serie de hiperparámetros que deben ajustarse para obtener el mejor rendimiento posible, como ocurre en cualquier problema de ML. Para su configuración se ha empleado la técnica de búsqueda exhaustiva *Grid Search*, implementada manualmente en ambos casos al no disponer de una implementación genérica compatible con los modelos utilizados.

La estrategia seguida ha consistido en identificar, a partir de la literatura y de experiencias previas en problemas similares que han comentado distintas personas en foros, un conjunto de valores de referencia para cada hiperparámetro. A partir de estos valores, se han definido rangos de búsqueda que incluyen el valor de referencia junto con variaciones por encima y por debajo, con el objetivo de ajustar cada parámetro al corpus y al problema concreto. Dado que ambos modelos emplean funciones de pérdida y algoritmos de optimización distintos, cada uno dispone de su propio conjunto de hiperparámetros a explorar.

10.2.1. Hiperparámetros de Word2Vec

Los hiperparámetros de Word2Vec pueden agruparse en tres categorías según el componente del modelo al que pertenecen: los propios de la arquitectura Skip-Gram, los relacionados con Negative Sampling y el correspondiente al optimizador.

En cuanto a los hiperparámetros de **Skip-Gram**, los dos parámetros ajustados son *vector_size* y *window*. El primero determina la dimensionalidad de los vectores de *embedding*, es decir, el número de características con el que se representa cada palabra. Su elección depende del tamaño del vocabulario, la riqueza semántica del corpus y los recursos computacionales disponibles. Dado que el vocabulario es de únicamente 13 palabras, no se requiere una representación de alta dimensionalidad para capturar las relaciones semánticas presentes. Por ello, se exploraron valores en el rango $\{2, 3, 4\}$. El valor seleccionado por el *Grid Search* fue *vector_size* = 2, lo cual resulta coherente con el reducido tamaño del vocabulario y presenta la ventaja adicional de permitir visualizar directamente los *embeddings* en el plano y realizar una validación manual sin necesidad de aplicar técnicas de reducción de dimensionalidad como PCA.

El segundo parámetro, *window*, define la distancia máxima entre la palabra central y las palabras de contexto consideradas durante el entrenamiento, tal y como se ilustra en la Figura 10.1. Los valores explorados fueron $\{1, 2, 3\}$, teniendo en cuenta que las frases del corpus tienen entre 3 y 4 palabras: con una ventana de tamaño 2, la palabra central puede relacionarse con todas las demás en una frase de 4 palabras, mientras que con tamaño 1, solo se considera la palabra adyacente. El valor seleccionado fue *window* = 1,

lo que implica que el modelo aprende exclusivamente a partir de pares de palabras contiguas, una restricción adecuada dada la brevedad de las frases del corpus.

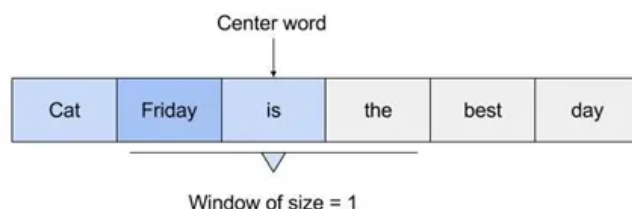


Figura 10.1: Ilustración del parámetro *window* en Skip-Gram: distancia máxima desde la palabra central hasta las palabras de contexto consideradas. Fuente: <https://medium.com/@zafaralibagh6/a-simple-word2vec-tutorial-61e64e38a6a1>.

Respecto a los hiperparámetros de **Negative Sampling**, los parámetros explorados son *negative* y *ns_exponent*. El primero indica el número de muestras negativas que se generan por cada par positivo durante el entrenamiento. El paper original que publicó Google, recomendó valores entre 5 y 20 (siendo 20 para corpus de gran escala). Dado que el corpus empleado cuenta únicamente con 13 palabras únicas, se optó por explorar valores en el rango $\{2, 3, 5, 6\}$, descartando valores superiores a la mitad del vocabulario para evitar que Negative Sampling degenerase en un comportamiento equivalente al softmax completo sobre todo el vocabulario, además de tener en consideración la recomendación del paper de Google. El valor seleccionado fue *negative* = 5, coincidiendo con el mínimo recomendado en el paper original.

El parámetro *ns_exponent* controla la distribución de probabilidad con la que se seleccionan las palabras negativas. Un valor de 1 desfavorece a las palabras poco frecuentes respecto a una distribución puramente proporcional a las frecuencias, mientras que un valor de 0,0 trata todas las palabras con igual probabilidad. El paper original de Word2Vec estableció 0,75 como valor fijo, sin embargo, un estudio posterior [5] argumentó que este parámetro debería adaptarse al problema concreto. Dado el notable desequilibrio léxico del corpus utilizado, se exploraron los valores $\{0,0, 0,3, 0,75\}$. El valor seleccionado fue *ns_exponent* = 0,0, lo que implica que todas las palabras tienen igual probabilidad de ser seleccionadas como muestras negativas, compensando en parte el desequilibrio de frecuencias del corpus.

Por último, el hiperparámetro relacionado con el **optimizador** es *alpha*, que establece la tasa de aprendizaje inicial del modelo. *Gensim* gestiona internamente la evolución de esta tasa a lo largo del entrenamiento, permitiendo únicamente fijar su valor inicial. La búsqueda de este parámetro se

realizó en dos fases iterativas. En una primera exploración se consideró el rango $\{0,1, 0,15, 0,175, 0,2\}$, con el objetivo de comprobar si valores pequeños eran suficientes para la convergencia. El valor seleccionado en esta fase fue 0,2, el límite superior del rango, lo que indicó que el modelo requería tasas de aprendizaje más elevadas. A partir de este resultado, se acotó el rango de búsqueda hacia valores superiores, explorando $\{0,2, 0,25, 0,275, 0,3\}$. En esta segunda fase el valor seleccionado fue $\alpha = 0,25$, situándose en el interior del rango y confirmando que la convergencia óptima se alcanza con tasas de aprendizaje relativamente elevadas.

10.2.2. Hiperparámetros de QWord2Vec

Los hiperparámetros del modelo cuántico pueden agruparse en tres categorías según el componente al que pertenecen: los propios del circuito, los relacionados con SPSA y los correspondientes al QNG. En la implementación utilizada, la clase que encapsula el algoritmo QN-SPSA integra los tres bloques en un único objeto de configuración.

En cuanto a los hiperparámetros del **circuito**, los dos parámetros ajustados son el número máximo de iteraciones y el número de muestras por simulación *shots*. Para el primero, se tuvo en cuenta que los circuitos variacionales cuánticos suelen necesitar muchas épocas para converger. El trabajo original de [24] empleó hasta 20.000 épocas. Dado que QN-SPSA está diseñado para converger en un número reducido de iteraciones, se consideró que 2500 iteraciones constituyen un límite suficientemente holgado, respaldado además por el mecanismo de *early stopping* descrito en la sección de entrenamiento. El valor del número de *shots* se fijó en 1024, un nivel elevado que reduce la varianza estadística de las distribuciones de probabilidad estimadas por el simulador y produce estimaciones más fiables de la función de pérdida.

Otro parámetro propio del circuito es el umbral A_{th} , que interviene en la fórmula heurística para estimar el número de capas L del circuito y representa la fracción máxima de datos de entrenamiento que se acepta perder. Su búsqueda se realizó en dos fases. En la primera exploración, y teniendo en cuenta que el paper de referencia de la heurística indicaba que los mejores valores de A_{th} se situaban en el intervalo $[0,01, 0,05]$, se evaluaron valores representativos dentro de dicho rango: $\{0,02, 0,03, 0,04, 0,05\}$. Los resultados de esta fase mostraron que los valores más altos (0,04–0,05) producen circuitos con muy pocas capas ($L \leq 6$), insuficientes para capturar la geometría del espacio de *embeddings*, mientras que los valores más bajos, especialmente 0,02 y 0,03, concentraban los mejores resultados. A partir de aquí, se acotó el rango de búsqueda en torno a dichos candidatos, explorando en la segunda fase los valores $\{0,018, 0,021, 0,024\}$. El valor seleccionado fue $A_{th} = 0,021$ (que coincide a su vez con el mismo valor del paper aunque se usaron métodos distintos para explorarlo). Añadido a A_{th} se encuentra

otra constante que es el valor $C = 2$ de la función de pérdida, este valor se ha utilizado el que indicaba el paper de referencia que mejor le fue para el corpus que se ha utilizado. Esto es porque el paper utiliza una función para calcular C en función de A_{th} , pero el A_{th} mejor encontrado por el grid search, coincidía con el mismo A_{th} del paper.

Respecto a los hiperparámetros de **SPSA**, los parámetros ajustados son los coeficientes iniciales de la tasa de aprendizaje a y de la perturbación c . Los exponentes de decaimiento $\alpha = 0,602$ y $\gamma = 0,101$ se tomaron directamente del paper original del algoritmo SPSA [33], dado que han sido validados matemáticamente y empíricamente por el autor y no requieren ajuste al problema concreto. Por el contrario, a y c sí dependen de las características de cada problema. Para estos dos parámetros se optó por valores moderadamente pequeños, por dos razones principales. En primer lugar, el optimizador se combina con una inicialización de los pesos basada en un *educated guess*. Esta técnica consiste en lo siguiente, antes de comenzar la optimización se evalúan 10 puntos de inicio aleatorios y se selecciona aquel con menor pérdida, lo que proporciona al optimizador un punto de partida ya orientado hacia una región prometedora del espacio de parámetros, sin necesidad de un paso de aprendizaje agresivo para explorar regiones lejanas. En segundo lugar, los valores de pérdidas de un circuito variacional cuántico sin pesos preentrenados, es especialmente ruidoso en las primeras iteraciones, debido a la alta varianza de las estimaciones obtenidas por muestreo finito del simulador, de modo que valores demasiado grandes de a o c podrían generar actualizaciones de parámetros inestables que alejasen al optimizador de la región de inicio favorable.

La búsqueda de a y c se realizó también en dos fases, de forma conjunta con el resto de hiperparámetros. En la primera exploración se utilizó un espacio amplio que incluía además los hiperparámetros de QNG, con el objetivo de identificar las regiones más prometedoras para todos los parámetros simultáneamente. A partir de los mejores resultados obtenidos, se realizó una segunda búsqueda más afinada con los mejores resultados obtenidos de la primera búsqueda. En esta segunda búsqueda, los valores candidatos propuestos han sido $a \in \{0,10, 0,16, 0,25\}$ y $c \in \{0,05, 0,07, 0,12\}$. Los valores seleccionados fueron $a = 0,16$ y $c = 0,07$. Adicionalmente, se activó el parámetro *blocking*, que restringe las actualizaciones del optimizador a aquellos casos en que la nueva pérdida no supere la anterior en más de una tolerancia fija, favoreciendo una convergencia más robusta.

Por último, los hiperparámetros relacionados con el **QNG** son la regularización del tensor métrico de Fubini-Study, el *hessian delay* y el *resampling*. Para su configuración se siguieron las recomendaciones disponibles en la literatura [7], que indican inicializar el tensor métrico de Fubini-Study como la matriz identidad en $g^{(0)}$. El coeficiente de regularización se ajustó teniendo en cuenta que un valor demasiado alto eliminaría la información del tensor métrico, mientras que un valor demasiado bajo provocaría su desva-

necimiento en regiones próximas a un mínimo. Los valores explorados en la primera fase del grid search fueron $\{10^{-4}, 5 \cdot 10^{-4}, 3 \cdot 10^{-3}\}$, y la segunda fase afinó la búsqueda en torno al mejor candidato, con el valor seleccionado de $1,6 \cdot 10^{-2}$. El *hessian delay* controla el número de iteraciones iniciales durante las cuales el tensor métrico no se aplica, evitando así que el ruido elevado de las primeras iteraciones distorsione la estimación del QNG. Los valores explorados incluyeron $\{200, 300, 400, 500, 700\}$, y el valor seleccionado fue 400. El *resampling* indica cuántas veces se ejecuta el cálculo del tensor métrico y se promedia el resultado, introduciendo mayor estabilidad a costa de un coste computacional adicional. Se fijó en un valor bajo (5) para mantener la viabilidad del experimento en un entorno de computación personal aunque esto supuso un aumento en el coste de computación algo considerable. Por último, dado que el corpus cuenta con 13 palabras y cada una tiene asignado un circuito con su propia codificación binaria en el espacio de Hilbert, se implementó una estrategia de *fidelidad rotante* para calcular el tensor métrico de Fubini-Study. En lugar de estimar siempre el tensor métrico sobre el mismo circuito, la fidelidad rota de forma uniforme a lo largo del vocabulario en cada llamada, de modo que el QNG recibe información de todas las palabras evitando así que perjudique a alguna de ellas en particular.

10.3. Entrenamiento

En esta sección se describe el proceso de entrenamiento seguido para cada modelo. Dado que Word2Vec y QWord2Vec emplean funciones de pérdida y algoritmos de optimización distintos, el análisis de cada uno se presenta de forma independiente.

10.3.1. Word2Vec

El proceso de entrenamiento de Word2Vec se basa en la función de pérdida de Negative Sampling como se explicó con anterioridad. Sin embargo, minimizar esta función auxiliar no garantiza directamente una buena calidad semántica de los *embeddings*. Por ello, se ha incorporado una métrica de evaluación adicional basada en el algoritmo *K-Means*, que permite cuantificar de forma directa la coherencia semántica de los vectores obtenidos.

El rol de cada componente es el siguiente. **Train loss** actúa como función de optimización, proporcionando la señal con la que el optimizador actualiza los parámetros en cada iteración. El **K-Means accuracy** constituye la métrica de calidad real de los *embeddings* y es la que determina el criterio de parada. Esta distinción es importante, usar el loss como criterio de *Early Stopping* podría interrumpir el entrenamiento en un punto subóptimo desde el punto de vista semántico, ya que el loss de Negative Sampling puede

seguir oscilando incluso cuando los *embeddings* han convergido a su mejor configuración.

Para calcular la precisión de *K-Means*, es necesario disponer de un agrupamiento (también conocido como clusters) de referencia (*ground truth*) de las palabras del vocabulario. Dicho agrupamiento fue generado mediante el modelo de lenguaje Gemini 3.0 Pro a partir del corpus, con la restricción de que los grupos fueran lo más equilibrados posible y sin poder aparecer la misma palabra en distintos grupos. El resultado fue posteriormente validado manualmente, obteniéndose los siguientes cuatro clústeres:

- **animal:** *dog, cat, animal, eyes*
- **food:** *apple, fish, milk*
- **culture:** *book, music, movie*
- **sentiment:** *i, like, hate*

Dado que *K-Means* asigna etiquetas arbitrarias a los clústeres, la precisión se calcula mediante el algoritmo húngaro, que encuentra la correspondencia óptima entre los clústeres predichos y los de referencia antes de evaluar el porcentaje de palabras correctamente asignadas.

Esta evaluación no se realiza en cada época, sino cada 200 épocas de entrenamiento. Esta frecuencia permite que el modelo evolucione suficientemente entre comprobaciones, manteniendo un equilibrio entre la sensibilidad de la métrica y la carga computacional adicional que supone ejecutar *K-Means* sobre los *embeddings* actuales.

La técnica de *Early Stopping* monitoriza el K-Means accuracy en cada comprobación y detiene el entrenamiento si no se supera el mejor valor previo en al menos un delta mínimo de 0,01 durante 5 comprobaciones consecutivas (paciencia = 5). En ese punto se recupera los pesos de los parámetros correspondiente al mejor accuracy registrado, reduciendo el riesgo de sobreajuste y garantizando que el modelo devuelto es el que mejor agrupa semánticamente las palabras.

Respecto a la partición de los datos, el entrenamiento se ha realizado con el 100% del corpus, sin reservar un conjunto de validación o prueba. Esta decisión fue tomada ya que uno de los objetivos del experimento de la versión clásica no es evaluar la capacidad de generalización del modelo a palabras nuevas, sino obtener *embeddings* de referencia para todas las palabras del vocabulario. Dado que Word2Vec solo genera un vector para una palabra si la ha observado durante el entrenamiento, excluir un subconjunto de frases implicaría que algunas palabras del vocabulario carecerían de embedding, dificultando la comparación con QWord2Vec, que requiere los vectores de referencia de todo el corpus.

En cuanto al flujo de entrenamiento, las frases del corpus se reordenan aleatoriamente antes del entrenamiento para evitar que el modelo aprenda

varias frases consecutivas cuyos valores están definidos en el mismo clúster, favoreciendo una exploración más amplia del espacio de parámetros. Se utiliza una semilla fija (valor 42) para garantizar la reproducibilidad de los resultados. El modelo se entrena desde cero utilizando los mejores hiperparámetros encontrados en el *Grid Search*. Una vez finalizado el entrenamiento, los *embeddings* de la mejor época encontrada, se exportan a un fichero de texto plano para su uso como referencia de comparación en la implementación cuántica.

La evolución del K-Means accuracy y del train loss delta a lo largo del entrenamiento se puede observar en la Figura 10.2.

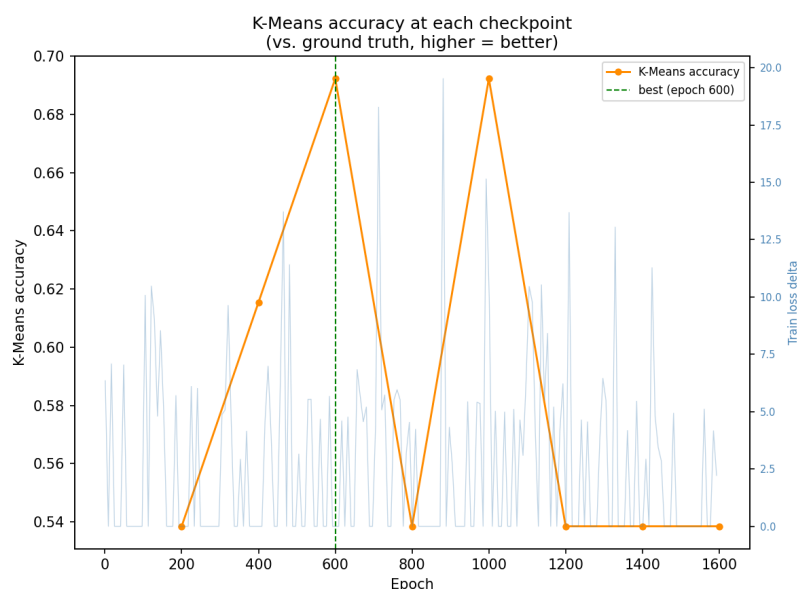


Figura 10.2: Evolución del K-Means accuracy (eje izquierdo, naranja) y del train loss delta (eje derecho, azul) a lo largo del entrenamiento de Word2Vec. La línea vertical verde indica la época del mejor punto registrado, punto en el que se activa el *Early Stopping*.

La gráfica ilustra con claridad por qué el train loss no es una métrica adecuada como criterio de parada. Desde las primeras iteraciones, el loss alcanza valores de 0 en varias ocasiones, lo que llevaría a detener el entrenamiento prematuramente si se usara como valor para medir la convergencia. Sin embargo, el K-Means accuracy sigue mejorando en ese mismo intervalo, lo que indica que los *embeddings* aún no han alcanzado su mejor configuración semántica en el espacio. El loss presenta oscilaciones continuas a lo largo de todo el entrenamiento, con valores que oscilan principalmente entre 0 y 7,5, lo cual es un comportamiento esperado en Negative Sampling dado su carácter estocástico a la hora de elegir candidatos de palabras negativas.

A partir de la época 600 se observa una caída pronunciada del K-Means accuracy acompañada de un pico del loss que supera el valor de 20. Tras este punto, el modelo recupera el mismo nivel de accuracy alcanzado en la época 600 en la siguiente comprobación. A continuación, el accuracy cae de forma definitiva y se estabiliza en su valor mínimo, lo que indica que el modelo ha entrado en una fase de sobreajuste en la que ya no es capaz de recuperar la configuración semántica óptima. Este comportamiento confirma que la época 600 corresponde a uno de los mejores puntos de entrenamiento, y que las iteraciones posteriores no aportan mejora alguna. El *Early Stopping* actúa correctamente al recuperar los valores de los embeddings de dicha época y detener el entrenamiento antes de que el modelo se siga degradando, reduciendo así el tiempo de entrenamiento principalmente.

10.3.2. QWord2Vec

El proceso de entrenamiento de QWord2Vec se basa en la función de pérdida personalizada descrita en el capítulo anterior, que combina la entropía cruzada con el coeficiente de correlación de Pearson. Sin embargo, esta función de pérdida no indica de forma directa si el modelo está prediciendo correctamente los contextos esperados para cada palabra. Por ello, al igual que en Word2Vec, se ha incorporado una métrica de evaluación adicional que permite cuantificar la calidad del modelo de forma más interpretable: el *error rate*.

Mientras que la **función de pérdida** actúa como señal de optimización, guiando al algoritmo QN-SPSA en la actualización de los parámetros del circuito en cada iteración. El **error rate** constituye la métrica de calidad real y es la que determina el criterio de parada. La función de pérdida puede oscilar o descender lentamente incluso cuando los pesos ya han alcanzado una configuración semánticamente adecuada, por lo que usarla directamente como criterio de parada podría llevar a detener el entrenamiento de forma prematura o excesivamente tardía.

La técnica de *Early Stopping* monitoriza el *error rate* cada 10 iteraciones y detiene el entrenamiento si no se supera el mejor valor previo en al menos un delta mínimo de 0,005 durante 10 comprobaciones consecutivas (paciencia = 10). La frecuencia de evaluación se ha fijado en 100 iteraciones, en lugar de evaluar en cada paso, con el objetivo de reducir el coste computacional. Esto se debe a que cada evaluación requiere ejecutar una simulación completa del circuito sobre todos los datos de entrenamiento, que es precisamente el cuello de botella del sistema. En el momento en que se activa la parada, se recuperan los pesos correspondientes al mejor *error rate* registrado, garantizando que el modelo devuelto es el que mejor predice los contextos esperados.

Respecto a la partición de los datos, el entrenamiento se ha realizado igualmente con el 100 % del corpus, por las mismas razones explicadas en

la sección de Word2Vec. Es necesario disponer de un *embedding* para cada palabra del vocabulario con el fin de poder realizar la comparación entre ambos modelos.

En cuanto al flujo de entrenamiento, antes de comenzar la optimización se lleva a cabo una inicialización de los parámetros del circuito ya que se está entrenando desde el inicio (*from scratch*). Se generan 10 puntos de inicio aleatorios y se evalúa la función de pérdida en cada uno de ellos, seleccionando aquel que presenta el valor más bajo como punto de partida para el optimizador. Esta estrategia reduce el riesgo de quedar atrapado en mínimos locales desde el inicio del entrenamiento obteniendo una ventaja en el inicio del entrenamiento.

A continuación, se definen dos funciones generadoras que adaptan dinámicamente, en cada iteración, la tasa de aprendizaje y la magnitud de la perturbación del algoritmo QN-SPSA siguiendo el esquema de decaimiento recomendado por los autores del algoritmo SPSA original. Sobre estas funciones se construye el optimizador con los mejores hiperparámetros encontrados durante el ajuste, descrito en la sección anterior. Adicionalmente, se ha activado el parámetro *blocking*, que restringe las actualizaciones de pesos a aquellos casos en que la nueva pérdida calculada no supere en más de el doble de desviación estandar del valor actual. Este mecanismo fuerza una convergencia más robusta, permitiendo al optimizador escapar de mínimos locales sin aceptar degradaciones excesivas de la función de pérdida.

Una vez finalizado el entrenamiento, se calcula el *error rate* final con los mejores pesos encontrados y se registra la evolución de la pérdida a lo largo de todas las iteraciones. La Figura 10.3 muestra dicha evolución junto con los valores de *error rate* y de pérdidas evaluados en cada comprobación.

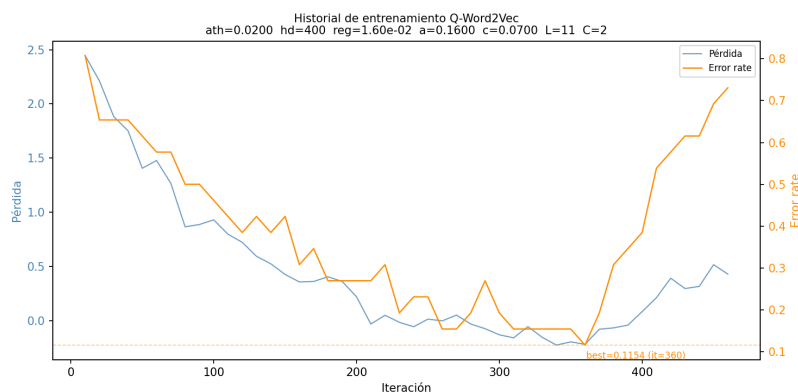


Figura 10.3: Evolución de la función de pérdida (eje izquierdo, azul) y del *error rate* (eje derecho, naranja) a lo largo del entrenamiento de QWord2Vec. La línea discontinua indica el mejor *error rate* registrado, punto en el que se activa el *Early Stopping*.

De aquí se pueden sacar bastantes conclusiones acerca de este algoritmo. La primera de ellas es relacionada a la pérdida. Tal y como se indicó en la definición de esta función de pérdida, viene controlada por un parámetro C que sirve para balancear. Como este parámetro multiplica a la entropía cruzada que se calcula entre los valores de embeddings de `qword2vec` y los training labels utilizados (recordar que tienen un valor de 0,5 dada la palabra de contexto, es decir las 2 palabras más cercanas a la palabra de referencia tiene valor de 0,5), Entonces al tener un loss negativo, significa que el modelo está aprendiendo una buena geometría de distancia entre las palabras de `Qword2vec` y las de entrenamiento. Por ello el modelo puede minimizar esta función de pérdida por debajo del 0 y esto es una buena señal ya que significa que el optimizador está haciendo su trabajo.

En cuanto al error rate, se puede apreciar cómo va disminuyendo a la par que la función de pérdida, esto significa que las 2 métricas son métricas muy buenas para calcular si el modelo está aprendiendo bien o no está aprendiendo. El error rate conseguido es muy bajo del 0,1, es decir, si le proporcionas una palabra de entrada de este corpus, con solo un 0,1 de probabilidad de fallo te va a decir las 2 palabras inmediatamente adyacentes que tiene (tamaño de ventana = 1) por lo que quiere decir que el modelo ha aprendido muy bien.

A partir de la época 360, se puede apreciar como la tendencia de la pérdida y el ratio de error pasa de ser convexo a ser cóncavo. Eso significa que el modelo ha llegado al máximo de lo que podía generalizar con el corpus que se tiene y ahora está sobreentrenando. Luego se puede apreciar como 100 iteraciones después salta el Early Stopping evitando así ejecutar más épocas de las necesarias.

Otra cosa importante que se ha visto es que en comparación al máximo de 20.000 épocas que le puso el autor original, con este algoritmo ha tardado 360 épocas en converger hasta el mejor mínimo encontrado. Esto indica como los hiperparámetros configurados para esta función de optimización han sido bastante buenos y se corresponde con la teoría de que QN-SPSA está diseñado para que converga en pocas iteraciones.

Además otro resultado curioso, es que tanto con el Grid Search como con la ejecución final con más iteraciones y shots, aún así ha convergido antes de que se aplique el tensor métrico de Fubini-Study, con esto quiero demostrar como para un problema de pequeñas dimensiones a pesar de que sobre el papel este algoritmo QN-SPSA ofrecía mejores resultados que el QNG o el SPSA por separado, al final para este dataset tan pequeño y con tan pocos qubits, ha resultado ser que la mejor opción encontrada en realidad era ejecutar SPSA finetuneando los valores de learning rate inicial, perturbación inicial y utilizando el parámetro de blocking. Lo más probable es que este algoritmo para un corpus más complejo y con mayores qubits a utilizar, se comportara mejor ya que tendría más parámetros que calcular.

10.4. Evaluación de los embeddings

10.4.1. Similitud semántica entre palabras

10.5. Análisis de tiempos de ejecución

Bibliografía

- [1] Felipe Almeida y Geraldo Xexéo. “Word Embeddings: A Survey”. En: *arXiv preprint* (2023). arXiv: 1901.09069 [cs.CL]. URL: <https://arxiv.org/abs/1901.09069>.
- [2] Frank Arute y col. “Quantum supremacy using a programmable superconducting processor”. En: *Nature* 574 (2019), págs. 505-510.
- [3] Marcello Benedetti y col. “Parameterized quantum circuits as machine learning models”. En: *Quantum Science and Technology* 4.4 (2019), pág. 043001. DOI: 10.1088/2058-9565/ab4eb5.
- [4] Jacob Biamonte y col. “Quantum machine learning”. En: *Nature* 549.7671 (2017), págs. 195-202. DOI: 10.1038/nature23474.
- [5] Hugo Caselles-Dupré, Florian Lesaint y Jimena Royo-Letelier. “Word2Vec applied to Recommendation: Hyperparameters Matter”. En: *CoRR* abs/1804.04212 (2018). arXiv: 1804.04212. URL: <http://arxiv.org/abs/1804.04212>.
- [6] David Castaño Bandera. 4.2. *El estado de un qubit y la esfera de Bloch*. Universidad de Málaga (SCBI). 28 de mayo de 2025. URL: https://www.scbi.uma.es/web/wp-content/uploads/Jupyterbook/CICC_UMA/Teoria/html/docs/Part_03/Chapter_004_02/Section_002_el_estado_de_un_qubit_y_la_esfera_de_bloch_myst.html (visitado 16-02-2026).
- [7] Yiheng Duan. *Quantum natural SPSSA optimizer*. PennyLane – Xanadu. 17 de jul. de 2022. URL: <https://pennylane.ai/qml/demos/qnspssa> (visitado 29-03-2026).
- [8] El Español. *¿Cuántas palabras tiene el español?* El Español. 2021. URL: https://www.elespanol.com/curiosidades/lenguaje/cuantas-palabras-tiene-espanol-castellano/641935856_0.html (visitado 25-02-2026).
- [9] Ferroviál. *¿Qué son los números complejos?* Ferroviál. 2026. URL: <https://www.ferrovial.com/es/stem/numeros-complejos/> (visitado 16-02-2026).

- [10] Noelia Freire. *El número e: de Euler a la vida cotidiana*. National Geographic España. 18 de ene. de 2024. URL: https://www.nationalgeographic.com.es/ciencia/numero-e-euler-a-vida-cotidiana_21407 (visitado 16-02-2026).
- [11] Julien Gacon y col. “Simultaneous Perturbation Stochastic Approximation of the Quantum Fisher Information”. En: *Quantum* 5 (20 de oct. de 2021), pág. 567. DOI: 10.22331/q-2021-10-20-567. URL: <https://quantum-journal.org/papers/q-2021-10-20-567/>.
- [12] GeeksforGeeks. *Phases of Natural Language Processing (NLP)*. GeeksforGeeks. 23 de jul. de 2025. URL: <https://www.geeksforgeeks.org/machine-learning/phases-of-natural-language-processing-nlp/> (visitado 16-02-2026).
- [13] Google. *word2vec*. Google Code Archive. 2013. URL: <https://code.google.com/archive/p/word2vec/> (visitado 26-02-2026).
- [14] Gopalan College of Engineering and Management. *Module 3: Quantum Computing & Quantum Gates*. Material de estudio. Applied Physics for Computer Science Engineering Stream. Gopalan College of Engineering y Management, 2022. URL: <https://www.gopalancolleges.com/gcem/pdf/syllabus/physics/cse/module-3-quantum-computing-quantum-gates.pdf> (visitado 16-02-2026).
- [15] Raffaele Guarasci, Giuseppe De Pietro y Massimo Esposito. “Quantum Natural Language Processing: Challenges and Opportunities”. En: *Applied Sciences* 12.11 (2022), pág. 5651. DOI: 10.3390/app12115651. URL: <https://www.mdpi.com/2076-3417/12/11/5651>.
- [16] Kevin Henner. *An intuitive introduction to text embeddings*. Stack Overflow. 9 de nov. de 2023. URL: <https://stackoverflow.blog/2023/11/09/an-intuitive-introduction-to-text-embeddings/> (visitado 23-02-2026).
- [17] IONOS. *Qubits: la base de los ordenadores cuánticos*. IONOS Digital Guide. 22 de feb. de 2023. URL: <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/qubits/> (visitado 16-02-2026).
- [18] Akbar Karimi. *NLP’s word2vec: Negative Sampling Explained*. Baeldung Computer Science. 26 de mar. de 2025. URL: <https://www.baeldung.com/cs/nlps-word2vec-negative-sampling> (visitado 03-03-2026).
- [19] Qiuchi Li y col. “Quantum-inspired complex word embedding”. En: *Proceedings of the Third Workshop on Representation Learning for NLP*. Association for Computational Linguistics, 2018, págs. 50-57. DOI: 10.18653/v1/W18-3006.

- [20] Inna Logunova y Olga Bolgurtseva. *Word2Vec: Why Do We Need Word Representations?* Serokell. URL: <https://serokell.io/blog/word2vec> (visitado 26-02-2026).
- [21] Chris McCormick. *Word2Vec Tutorial - The Skip-Gram Model*. McCormick ML. 19 de abr. de 2016. URL: <https://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/> (visitado 26-02-2026).
- [22] Chris McCormick. *Word2Vec Tutorial Part 2 - Negative Sampling*. McCormick ML. 11 de ene. de 2017. URL: <https://mccormickml.com/2017/01/11/word2vec-tutorial-part-2-negative-sampling/> (visitado 26-02-2026).
- [23] Tomas Mikolov y col. "Distributed Representations of Words and Phrases and their Compositionality". En: *arXiv preprint* (2013). arXiv: 1310.4546 [cs.CL]. URL: <https://arxiv.org/abs/1310.4546>.
- [24] Hiroshi Ohno. "Quantum circuits for word embeddings". En: *Quantum Machine Intelligence* (2025). DOI: 10.1007/s42484-025-00309-w. URL: <https://doi.org/10.1007/s42484-025-00309-w>.
- [25] PennyLane. *Variational circuits — PennyLane*. PennyLane. URL: https://pennylane.ai/qml/glossary/variational_circuit (visitado 21-02-2026).
- [26] Rod Pierce. *Bra-Ket Notation*. MathsIsFun. 2025. URL: <https://www.mathsisfun.com/physics/bra-ket-notation.html> (visitado 16-02-2026).
- [27] Rod Pierce. *Fórmula de Euler para Números Complejos*. Disfruta Las Matemáticas. 2024. URL: <https://www.disfrutalasmatematicas.com/algebra/formula-euler.html> (visitado 16-02-2026).
- [28] Spencer Porter. *Understanding Cosine Similarity and Word Embeddings*. Medium. 28 de ago. de 2023. URL: <https://spencerporter2.medium.com/understanding-cosine-similarity-and-word-embeddings-dbf19362a3c> (visitado 25-02-2026).
- [29] Sangaku S.L. *Representación de números complejos en el plano*. Sangaku Maths. 2026. URL: <https://www.sangakoo.com/es/temas/representacion-de-numeros-complejos-en-el-plano> (visitado 16-02-2026).
- [30] Freya Shah. *Quantum Encoding: An Overview*. Quantum Zeitgeist. 2021. URL: <https://quantumzeitgeist.com/quantum-encoding-an-overview/> (visitado 18-02-2026).
- [31] Sam Sholtis. *Here's a better way to measure atomic qubits*. Futurity. 26 de mar. de 2019. URL: <https://www.futurity.org/quantum-computing-2018412/> (visitado 16-02-2026).

- [32] Sukin Sim, Peter D Johnson y Alán Aspuru-Guzik. “Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms”. En: *Advanced Quantum Technologies* 2.12 (2019), pág. 1900070.
- [33] J. C. Spall. “Implementation of the simultaneous perturbation algorithm for stochastic optimization”. En: *IEEE Transactions on Aerospace and Electronic Systems* 34.3 (jul. de 1998), págs. 817-823. DOI: 10.1109/7.705889.
- [34] SpinQ Technology. *Ultimate Guide to Quantum Gates: Everything You Need to Know*. 14 de mar. de 2025. URL: <https://www.spinquantum.com/news-detail/quantum-gates> (visitado 16-02-2026).
- [35] James Stokes y col. “Quantum Natural Gradient”. En: *Quantum* 4 (2020), pág. 269. DOI: 10.22331/q-2020-05-25-269. URL: <https://quantum-journal.org/papers/q-2020-05-25-269/>.
- [36] Cole Stryker y Jim Holdsworth. *What is NLP (Natural Language Processing)?* IBM. URL: <https://www.ibm.com/think/topics/natural-language-processing> (visitado 16-02-2026).
- [37] The MITRE corporation. *The Bloch Sphere*. Intro to Quantum Software Development. 1 de jul. de 2022. URL: <https://stem.mitre.org/quantum/quantum-concepts/bloch-sphere.html> (visitado 16-02-2026).
- [38] Sik-Ho Tsang. *Review: Distributed Representations of Words and Phrases and their Compositionality — Negative Sampling (NLP)*. Medium. URL: <https://sh-tsang.medium.com/review-distributed-representations-of-words-and-phrases-and-their-compositionality-negative-ab9ebfc3f041> (visitado 01-03-2026).
- [39] Charles M. Varmantchaonala y col. “Quantum Natural Language Processing: A Comprehensive Survey”. En: *IEEE Access* 12 (2024), págs. 99578-99598. DOI: 10.1109/ACCESS.2024.3420707.
- [40] Dominic Widdows y col. “Quantum Natural Language Processing”. En: *KI - Künstliche Intelligenz* 38.4 (dic. de 2024), págs. 293-310. DOI: 10.1007/s13218-024-00861-w. URL: <https://doi.org/10.1007/s13218-024-00861-w>.
- [41] Wikipedia contributors. *Simultaneous perturbation stochastic approximation*. Wikipedia. URL: https://en.wikipedia.org/wiki/Simultaneous_perturbation_stochastic_approximation (visitado 29-03-2026).

- [42] Peter Wittek y col. “Combining Word Semantics within Complex Hilbert Space for Information Retrieval”. En: *Quantum Interaction*. Vol. 8369. Lecture Notes in Computer Science. Springer, 2014, págs. 160-171. URL: <https://www.diva-portal.org/smash/get/diva2:887708/FULLTEXT01.pdf>.
- [43] Yanying Wu y Quanlong Wang. *A Categorical Compositional Distributional Modelling for the Language of Life*. 2019. arXiv: 1902.09303 [q-bio.QM]. URL: <https://arxiv.org/abs/1902.09303>.
- [44] Meng Xie y col. “Modeling quantum entanglements in quantum language models”. En: *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI’15)*. AAAI Press, 2015, págs. 1362-1368.
- [45] Peng Zhang y col. “End-to-end quantum-like language models with application to question answering”. En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI’18)*. AAAI Press, 2018.
- [46] Han-Sen Zhong y col. “Quantum computational advantage using photons”. En: *Science* 370 (2020), págs. 1460-1463.

Glosario

DVS	Descomposición en Valores Singulares.
GD	Gradient Descent.
GPT	Generative Pre-trained Transformer.
IA	Inteligencia Artificial.
LLM	Large Language Model.
ML	Machine Learning.
MNL	Modelo Neuronal del Lenguaje.
NER	Named Entity Recognition.
NISQ	Noisy Intermediate-Scale Quantum.
NLP	Natural Language Processing.
PLN	Procesamiento del Lenguaje Natural.
CBOW	Continuous Bag of Words.
CQM	Categorical Quantum Mechanics.
CVC	Circuitos Variacionales Cuánticos.
DisCoCat	Distributional Compositional Categorical.
DL	Deep Learning.
DPT	Dependency Parse Tree.

PLNC Procesamiento del Lenguaje Natural Cuántico.	QNG Quantum Natural Gradient Descent.
POS Part-of-Speech.	Qubit Quantum Bit.
QML Quantum Machine Learning.	SPSA Simultaneous Perturbation Stochastic Approximation.
QN-SPSA Quantum Natural Simultaneous Perturbation Stochastic Approximation.	WSD Word Sense Disambiguation.